EAST WEST UNIVERSITY

Automated Fabrics Defect Detection Using Computer Vision and Deep Learning

Submitted from

Sumaiya Hoque Riyan

2022-1-60-111

Jahir Hasan Biddut

2022-1-60-096

Md. Nahid Hasan

2022-1-60-221

Md. Momtajur Rahman

2022-1-60-370

Supervised By

Md Sabbir Hossain

A capstone project report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering



Department of Computer Science and Engineering

East West University

Dhaka-1212, Bangladesh

13 August, 2026

Declaration

We, **Sumaiya Hoque Riyan, Jahir Hasan Biddut, Md. Nahid Hasan and Md. Momtajur Rahman** hereby, declare that the work presented in this capstone project report is the outcome of the investigation performed by us under the supervision of **Md. Sabbir Hossain** ,Lecturer, Department of Computer Science and engineering, East West University. We also declare that no part of this project has been or is being submitted elsewhere for the award of any degree or diploma, except for publication.

Signature

Countersigned

……

Md. Sabbir Hossain
Supervisor

……..…
(Sumaiya Hoque Riyan)
(ID: 2022-1-60-111)

……..…
(Jahir Hasan Biddut)
(ID: 2022-1-60-096)

……..…
(Md. Nahid Hasan)
(ID: 2022-1-60- 221)

……..…
(Md. Momtajur Rahman)
(ID: 2022-1-60- 370)

Letter of Acceptance

The capstone project titled "**Automated Fabrics Defect Detection Using Computer Vision and Deep Learning**", submitted by Sumaiya Hoque Riyan, Jahir Hasan Biddut, Md. Nahid Hasan and Md. Momtajur Rahman from the Department of Computer Science and Engineering has been examined. Upon the examination committee's recommendation, we approve the project and report, as they meet the requirements for partial fulfillment of the Bachelor of Science in Computer Science and Engineering degree.

Supervisor

……

(Md Sabbir Hossain)

Lecturer, Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

Chairperson

……

(Dr. Maheen Islam)

Associate Professor, Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

Acknowledgment

Our study of 'Automated Garments Defect Detection Using Computer Vision and Deep Learning' has been a great pleasure. Throughout this project, we have accumulated appropriate expertise and experience.

We would like to express our deep and sincere gratitude to our research supervisor, Md Sabbir Hossain, for allowing us to conduct research and providing invaluable guidance throughout this work. His dynamism, vision, sincerity, and motivation have deeply inspired us. He has taught us the methodology to carry out the work and to present the works as clearly as possible. It was a great privilege and honor to work and study under his guidance. We would like to thank him for his friendship, empathy, and great sense of humor.

We are greatly indebted to our honorable teachers of the Department of Computer Science and Engineering at the East West University who taught us during our study. Without any doubt, their teaching and guidance have completely transformed us to the people that we are today.

We are extremely thankful to our parents for their unconditional love, endless prayers, care, and immense sacrifices for educating and preparing us for our future. We would like to say thanks to our friends and relatives for their kind support and care.

Finally, we would like to thank all the people who have supported us to complete the project work directly or indirectly.

Sumaiya Hoque Riyan
August, 2026

Jahir Hasan Biddut
August, 2026

Md. Momtajur Rahman
August , 2026

Md. Nahid Hasan
August, 2026

Abstract

Automated garment defect detection plays a crucial role in improving product quality and reducing manual inspection costs in the textile manufacturing industry. This study proposes a computer vision-based automated defect detection framework using lightweight YOLO object detection models. Four state-of-the-art architectures, namely YOLOv8s, YOLOv9s, YOLOv11s, and YOLO26s, were trained and evaluated on a garment defect dataset containing five classes: defect-free, hole, knot slub, spot, and yarn. Model performance was assessed using Precision, Recall, F1-score, mean Average Precision at IoU 0.50 (mAP@50), and mean Average Precision across multiple IoU thresholds (mAP@50:95). Experimental results demonstrate that YOLOv8s achieved the best overall performance, with a Precision of 96.27%, Recall of 94.28%, F1-score of 95.27%, mAP@50 of 98.56%, and mAP@50:95 of 75.83%. YOLO26s also demonstrated competitive performance while requiring fewer training epochs, indicating improved training efficiency. Comparative analysis further revealed that YOLOv8s provides the best balance between detection accuracy, localization capability, and computational efficiency for automated garment inspection. The proposed framework offers an effective and practical solution for intelligent textile quality control, with significant potential for real-time industrial deployment.

Keywords: Computer Vision; Deep Learning; Garment Defect Detection; YOLO Object Detection; Textile Quality Inspection.

Table of Contents

Declaration ..... **i**

Letter of Acceptance ..... **ii**

Acknowledgment ..... **iii**

Abstract ..... **iv**

Table of Contents ..... **v**

List of Figures ..... **viii**

List of Tables ..... **ix**

Chapter 1

Introduction **1**

1.1 Background Study **1**

1.2 Problem Statement **2**

1.3 Research Questions **3**

1.4 Research Objectives **4**

Chapter 2

Background and Motivation **5**

2.1 Background **5**

2.2 Motivation **6**

2.3 Problem Analysis 10

2.4 Requirements Engineering 11

2.4.1 Functional Requirements 11

2.4.2 Non-Functional Requirements 12

2.5 Business Model 12

2.6 Social, Economic, and Environmental Impact 13

Chapter 3

Methodology 16

Chapter 4

Design and Implementation 24

4.1 Dataset Preparation 24

4.2 Preprocessing 27

4.2.1 Input Image Size 27

4.2.2 Data Augmentation 28

4.2.3 Normalization 29

4.2.4 Albumentations Integration 29

4.3 Models 29

4.4 Model Evaluation 32

4.4.1 YOLOv8s 32

4.4.2 YOLOv9s 37

4.4.3 YOLOv11s 42

4.4.4 YOLO26s 48

4.5 Prototype Web Application.....52

4.6 FabDetector Application — UML Diagrams 55

4.6.1 Use Case Diagram 55

4.6.2 Activity Diagram.....56

4.6.3 Class Diagram 57

4.6.4 ER Diagram.....58

Chapter 5

Results & Discussion.....60

5.1 Overall Performance Comparison 60

5.2 Comparative Analysis.....60

5.3 Qualitative and Quantitative Performance Analysis of YOLOv8s 61

5.4 Discussion.....65

Chapter 6

Conclusion and Limitations 66

Limitations.....67

Chapter 7

Future Work.....68

Bibliography 69

Appendix.....71

Appendix 1.....71

List of Figures

Figure 1. Workflow of the proposed fabric defect detection system. 11
Figure 2. Some samples of dataset. 16
Figure 3. YOLOv8s Training and Validation Loss and Validation Accuracy... 22
Figure 4. YOLOv8s Per-class metrics.. 23
Figure 5. YOLOv8s Confusion Matrix.. 25
Figure 6. YOLOv8s Training metrics and loss curves 26
Figure 7. YOLOv8s Training and Validation Loss and Validation Accuracy... 27
Figure 8. YOLOv9s Per-class metrics.. 28
Figure 9. YOLOv9s Confusion Matrix.. 30
Figure 10. YOLOv9s Training metrics and loss curves.. 32
Figure 11. YOLOv11s Training and Validation Loss and Validation Accuracy... 33
Figure 12. YOLOv11s Per-class metrics.. 34
Figure 13. YOLOv11s Confusion Matrix.. 36
Figure 14. YOLOv11s Training metrics and loss curves.. 38
Figure 15. YOLO26s Training and Validation Loss and Validation Accuracy... 39
Figure 16. YOLO26s Per-class metrics.. 40
Figure 17. YOLO26s Confusion Matrix.. 42
Figure 18. YOLO26s Training metrics and loss curves... 43
Figure 19. Prototype web application for automated garment defect detection.... 45
Figure 20. Illustrates the comparative performance of the four models.... 47
Figure 21. Illustrates the comparative performance of the four models in line charts .48
Figure 22. The comprehensive performance evaluation of the YOLOv8s model... 49
Figure 23. YOLOv8s successfully detects multiple garment defect categories ... .50
Figure 24. YOLOv8s normalized confusion matrix 51

List of Tables

Table 1. Comparison Table of Existing Studies in Fabric Defect Detection.....9
Table 2. Architectural Comparison of the Evaluated YOLO Models.. 20
Table 3. YOLOv8s Per-Class Performance values.....24
Table 4. YOLOv9s Per-Class Performance values.....28
Table 5. YOLOv11s Per-Class Performance values.....34
Table 6. YOLO26s Per-Class Performance values.....40
Table 7. Features of the Prototype. 45
Table 8. Performance Comparison of YOLO Models. 46

Chapter 1

Introduction

1.1 Background study

The garment and textile industry relies heavily on strict quality control because even small surface defects can significantly impact customer satisfaction, export value, and overall production efficiency. Minor imperfections such as holes, knot or slub irregularities, stains, spots, tears, and yarn faults may seem insignificant at first glance, but they can render an entire product unacceptable in the market. This is especially critical in export-oriented production, where buyers maintain very high-quality standards.

Despite the importance of quality assurance, many factories still depend largely on manual inspection. Human inspectors can identify obvious defects effectively; however, their performance is not always consistent. Factors such as long working hours, fatigue, poor lighting conditions, and the repetitive nature of fabric patterns can reduce accuracy over time. As production speeds continue to increase to meet global demand, relying solely on human inspection becomes both inefficient and unreliable.

To address these challenges, computer vision offers a powerful and scalable alternative. By automatically analyzing image data, computer vision systems can detect defects more consistently and at a much faster rate than manual methods. When integrated with deep learning techniques, these systems become even more capable—they can not only detect defects but also classify them into different categories and highlight their exact locations. Additionally, they provide visual evidence, which can be useful for quality verification and process improvement.

Among various computer vision approaches, object detection is particularly well-suited for this task. Unlike simple classification methods, object detection identifies both *what* the defect is and *where* it is located on the fabric. This makes it highly practical for real-world industrial applications.

This project aims to develop an automated system for detecting garment defects using the YOLOv8n object detection framework. The model is trained to classify images into two categories:

- Defect-Free
- Defect

In this work, garment defect detection is formulated as a multi-class object detection problem. The system is implemented using YOLOv8n, a lightweight and efficient one-stage object detection model. YOLOv8n is chosen because it offers a strong balance between computational efficiency and detection performance, making it suitable for deployment in real production environments where speed and accuracy are both critical.

1.2 Problem Statement

The main problem in this project is to automatically find and locate defects in garment images. It is not enough to just say whether a defect is present or not. The system also needs to show where the defect is on the fabric and identify what type of defect it is. This makes the task more complex.

This is challenging because defects can look very different from each other. They vary in size, shape, color, and texture. Some defects, like small spots or yarn faults, are very hard to see because they look similar to the fabric pattern. Even humans can miss these. Other defects, like holes or tears, are easier to notice, but they can appear in different sizes, positions, or angles, which makes detection harder for a model.

Another difficulty comes from the fabric itself. Many fabrics have repeated patterns or complex designs. These patterns can confuse the system, making it think a normal design is a defect or missing an actual defect.

There are also data-related problems. Some types of defects appear very rarely in the dataset. Because of this, the model may not learn them well and might fail to detect them later. Also, collecting and labeling defect images takes a lot of time and effort, so the amount of training data is often limited. This makes it harder for the model to perform well on new or unseen images.

Overall, detecting garment defects is not easy. The system needs to handle different types of defects, complex fabric patterns, and limited data while still giving accurate results.

Therefore, there is a need for an automated, accurate, and efficient defect detection system that can:

- Detect the presence of defects in garment images
- Distinguish between defect-free and defective fabrics
- Localize defects within the image for better analysis
- Operate in near real-time for industrial applications

The challenge lies in developing a system that can handle:

- Variability in defect types, sizes, and textures
- Imbalanced datasets
- Subtle and small defects that are difficult to detect
- Complex fabric patterns

To address these challenges, this project proposes the use of YOLOv8-based object detection models to build a robust system capable of accurate and efficient garment defect detection.

1.3 Research Questions

This study is further guided by these key research questions:

1. How can a deep-learning-based object detector be used to automatically identify and localize garment defects from images?
2. What does the submitted six-class garment defect dataset reveal about split distribution, annotation density, and class imbalance?
3. What level of performance can a lightweight YOLOv8n model achieve on this dataset in terms of precision, recall, and mean average precision?
4. Which defect categories are easiest and hardest to detect, and what limitations explain the observed errors?
5. How can deep learning models detect and localize garment defects?
6. What is the performance of YOLOv8 models on this dataset?
7. Which defects are easier or harder to detect?
8. How does dataset imbalance affect performance?

1.4 Research Objectives

The primary objective of this research is to develop an automated fabric defect detection system using deep learning models trained on image datasets. This work aims to evaluate and compare the performance of three state-of-the-art architectures YOLOv8s , YOLOv9s , YOLOv11s and YOLOv26s to determine the most effective model for accurate classification. The study further seeks to design a complete preprocessing and training pipeline, improve classification accuracy, and reduce manual inspection effort. Ultimately, the goal is to produce a high-performing model that can be scaled for industrial deployment, enabling faster, more reliable, and more consistent identification of textile materials.

Chapter 2

Background and Motivation

This chapter presents the background of fabric defect detection using machine learning. It introduces the fundamental concepts of deep learning, including techniques such as transfer learning and modern architectures, which are essential for image-based fabric defect detection. The chapter also reviews various image classification models and explores how machine learning approaches are applied to material and textile defect detection tasks.

2.1 Background

Textile and garment manufacturing rely heavily on maintaining consistent surface quality, because even small defects can reduce the value of the final product. As highlighted by Rasheed et al., fabric quality has a direct impact on market acceptance, and when defects are found, production often has to pause so the issue can be checked, recorded, and fixed. This not only slows down the workflow but also increases production costs and reduces efficiency [1].

The same study points out that there are more than seventy different types of fabric defects reported in the literature. Some of the most common ones include holes, stains, broken threads, thick lines (bars), broken weft, and incorrect weaving patterns [1]. This

shows that defect detection is not a simple task. A real inspection system must be able to handle a wide variety of defect types. Some defects are easy to see, but many are small, low-contrast, or hidden within the natural texture and pattern of the fabric, making them difficult to detect.

The problem becomes even more complex in garment inspection compared to raw fabric inspection. In real factory environments, the system often works with finished or partially finished garments instead of flat, controlled fabric samples. This introduces additional challenges such as folds, wrinkles, stitching variations, inconsistent lighting, and background noise. These factors can confuse both human inspectors and automated systems.

According to Deep-GD, this issue is very important in industrial settings. If defects are not detected early, they can lead to wasted materials, lower product quality, and extra rework during later stages of production [2]. This not only increases costs but also affects delivery time and customer satisfaction. Therefore, building an accurate and reliable defect detection system is essential for improving efficiency and maintaining high-quality standards in the garment industry.

- Consistent & Precise: Delivers objective, repeatable results 24/7 for superior quality assurance.
- Rapid & Scalable: Inspects fabrics at high speed, seamlessly integrating into production lines.
- Advanced Capabilities: Not only detects but also classifies and localizes defects, providing actionable insights.

2.2 Motivation

One of the main reasons for using automation in garment inspection is consistency. Human inspectors can become tired, lose focus, or make mistakes after working for long hours. Their judgment may also change depending on lighting conditions or

workload. In contrast, a deep learning model does not get tired or distracted. It analyzes every image using the same rules and criteria, which leads to more stable and reliable results over time.

The second important reason is speed. In modern factories, production lines move very fast, and manual inspection can become a bottleneck. A camera combined with a trained model can quickly process a large number of images without needing to pause for human decisions. This allows defects to be detected in real time, helping factories maintain high production speed without compromising quality.

The third advantage is traceability. When inspection is automated, all results can be recorded digitally. This means images of defects, model confidence scores, and defect categories can be stored for future use. These records are very useful for auditing, analyzing production problems, and improving the manufacturing process. For example, if a certain defect keeps appearing, managers can trace it back to its source and fix the issue more effectively.

Beyond the factory, automated quality assessment also has value for consumers. For example, the IJNRD paper highlights that automated systems can help buyers better understand the condition and quality of garments before purchase [3]. Although this study is not as technically detailed as modern deep learning research, it shows that there is real demand for such systems even outside industrial environments. This suggests that automated garment inspection can play an important role not only in production but also in improving customer confidence and transparency in the marketplace.

Even though deep learning shows great potential, automatically detecting garment defects is still a challenging task. There are several practical difficulties that make this problem complex:

• **Texture complexity:**

Many fabrics have repeated patterns or knitted designs. These patterns can make it hard to notice small defects because they blend into the background. Sometimes, what looks like a defect might actually be part of the design, and vice versa.

• **Class imbalance:**

In most datasets, some types of defects appear very often, while others are rare. Because of this, the model learns the common defects better but struggles to recognize the rare ones. This leads to uneven performance across different defect types.

• **Small-object sensitivity:**

Some defects, such as tiny holes, yarn faults, or light stains, are very small and occupy only a small area of the image. Detecting these requires the model to be very precise, which is often difficult, especially when image resolution or quality is limited.

• **Annotation quality:**

The quality of labeled data is very important for training. If the annotations are inconsistent, incorrect, or mixed (for example, combining bounding boxes with segmentation masks without clear rules), it can confuse the model and reduce its performance.

• **Deployment constraints:**

A model that works well during training on a powerful GPU may not perform the same way in real production. In factories, systems often need to run on edge devices or lower-power hardware. This means the model must be optimized or compressed without losing too much accuracy, which adds another layer of difficulty.

• **Limited Training Data**:

The overall dataset size is relatively small compared to typical deep learning requirements. With limited data, the model may not be exposed to all possible variations of defects, such as different lighting conditions, angles, or fabric types. This restricts the model's ability to generalize to unseen data and increases the risk of overfitting, where the model performs well on training data but poorly on new images.

Table 1. Comparison Table of Existing Studies in Fabric Defect Detection.

Study / Source	Method	Strengths	Limitations
Deep learning survey [1]	Faster R-CNN, Mobile-UNet, YOLO-LFD, DCGAN	Better localization and learned feature extraction	High data demand, training cost, limited public datasets
IJNRD paper [3]	CNN + GUI-oriented quality identification	Consumer-oriented, simple garment quality analysis	Less rigorous methodology, weak industrial grounding
Deep-GD [2]	Bilateral filter + SE-Net + RDF-BO	High accuracy (97.16%), strong hybrid design	Complex pipeline, not lightweight or real-time focused
This capstone project [4]	YOLOv8n object detection	Simple end-to-end detection, fast, supports localization + classification	Moderate performance, affected by data imbalance/labeling
Lee et al. (2009) [5]	Convolutional Deep Belief Networks	Learns hierarchical features automatically	Early model, less efficient, not real-time focused
MATLAB Expo (2018) [6]	Deep learning in MATLAB	Easy to use, beginner-friendly	Limited scalability for industrial systems
MATLAB E-Book [7]	Educational workflows	Good conceptual understanding	Mostly theoretical, not application-specific
CompleteGate CNN [8]	CNN evolution (LeNet, AlexNet)	Clear explanation of model development	No experimental validation
ImageNet (2010) [9]	Large-scale classification benchmark	Enabled major deep learning progress	Classification only, no detection/localization
Fei-Fei Li Lecture [10]	CNN architecture analysis	Strong theoretical insights	Not application-specific

Study / Source	Method	Strengths	Limitations
Sánchez & Perronnin [11]	Feature compression	Efficient for large datasets	Depends on handcrafted features
MATLAB Vision [12]	Image classification pipeline	Simple implementation	Limited industrial use
AlexNet (2015) [13]	Deep CNN classification	Breakthrough accuracy, strong feature learning	High compute cost, no localization
MATLAB DL Guide [14]	Neural network classification	Structured workflow, easy to follow	Not flexible for detection tasks
Kishore et al. (2013) [15]	ANN with shape + texture features	Combines multiple features effectively	Domain-specific (gesture recognition)
Kishore et al. (2015) [16]	Multi-camera + Fourier + ANN	Improved recognition using multi-view data	Complex setup, not for industrial inspection

2.3 Problem Analysis

2.3.1 Technical Problem Decomposition

The overall problem of garment defect detection can be broken down into five connected parts, each of which plays an important role in building a reliable system.

The first step is **image acquisition**. This means collecting images that truly represent real production conditions. In factories, garments are not always perfectly flat or well-lit. There may be folds, wrinkles, shadows, or background noise. If the images used for training do not reflect these real conditions, the model may perform well in testing but fail in actual use.

The second step is **annotation quality**. This involves labeling the images correctly. Each defect must be marked accurately with the right class and location. If labels are missing, incorrect, or inconsistent, the model will learn the wrong patterns. Good annotation is very important because the model completely depends on this data during training.

The third part is **feature learning**. Here, the model tries to learn the difference between normal fabric and defective areas. This is challenging because some defects are very subtle and look similar to the fabric's natural texture. The model must learn to focus on small details, patterns, and irregularities that indicate a defect.

The fourth step is **detection and classification**. After learning useful features, the model needs to apply them to new images. It must correctly find where the defect is (localization) and identify what type of defect it is (classification). This requires both accuracy and precision, especially when defects are small or appear in different shapes and positions.

The final step is **decision support**. This is about how the results are presented and used in practice. The system should provide clear outputs, such as bounding boxes, defect labels, and confidence scores, so that quality-control teams can easily understand and act on the results. For example, they may use this information to reject faulty items, track defect trends, or improve the production process.

All five parts are closely connected. If one part is weak—such as poor image quality or incorrect labeling—the overall system performance will suffer. Therefore, building an effective garment defect detection system requires careful attention to each stage, from data collection to final decision-making.

2.4 Requirements Engineering

2.4.1 Functional Requirements

These describe what the system needs to actually do, split roughly by pipeline stage:

ID	Requirement	Description
FR1	**Data Collection**	The system shall accept garment images from a dataset folder (YOLO-format, organized by class) or a live capture source for inspection, resuming from where a previous

ID	Requirement	Description
		ingestion run left off rather than re-processing already-loaded images.
FR2	**Dataset Configuration**	The system shall read class definitions and train/validation/test split paths from a YAML configuration file, and support the five original classes (defect_free, hole, knot_slub, spot, yarn_defect) as well as the binary regrouping (defect_free vs. defect).
FR3	**Preprocessing & Augmentation**	The system shall resize input images to the model's required input size (640×640 for YOLO training), normalize pixel values, and apply augmentation (rotation, shift, shear, zoom, horizontal flip) during training to improve generalization.
FR4	**Defect Detection**	The model shall detect defect regions using bounding boxes and assign a class label to each detected region, distinguishing defect-free garments from defective ones.
FR5	**Annotated Output Generation**	The system shall generate annotated output images that overlay predicted bounding boxes, class labels, and confidence scores onto the original garment image.
FR6	**Model Evaluation**	The training pipeline shall report evaluation metrics — Precision, Recall, F1-score, mAP@0.50, and mAP@0.50:0.95 — for each trained model (YOLOv8s, YOLOv9s, YOLOv11s, YOLO26s) to support comparison.
FR7	**Batch & Single Inference**	The implementation shall support batch inference over a test directory of images as well as single-image prediction on demand.

ID	Requirement	Description
FR8	**Web Application Upload**	The system shall provide a web interface where users can upload one or more images for defect analysis and receive detection results.
FR9	**Video Stream Inference**	The system shall support processing video footage to detect defects continuously in near real-time, in addition to static image uploads.

Non-Functional Requirements-

ID	Requirement	Description
NFR1	**Accuracy / Reliability**	The best-performing model (YOLOv8s) should sustain the reported performance level (~96% Precision, ~94% Recall, ~98.5% mAP@50) on new, unseen garment images from similar factory conditions.
NFR2	**Performance**	Inference on a single uploaded image or video frame should return detection results within a few seconds to support near real-time use on a factory floor.
NFR3	**Usability**	The web interface should be simple enough for a non-technical quality-control worker to upload an image or start a video feed without needing training.
NFR4	**Robustness to Class Imbalance**	The system should remain reasonably accurate on less-frequent defect types despite the dataset's class imbalance (e.g., spot and defect_free being more common than others).
NFR5	**Scalability**	The detection pipeline should remain responsive as more garment images or longer video streams are processed, without requiring architecture changes.

ID	Requirement	Description
NFR6	**Deployability / Cost**	The system should be deployable within constrained compute and budget conditions (the project was built end-to-end for under Tk 10,000 using free-tier compute/hosting), and should be feasible to run on lower-power or edge hardware for factory use.
NFR7	**Maintainability**	Pipeline stages (data collection, preprocessing, training, evaluation, web app) should be independently testable and re-runnable, so a new model (e.g., a future YOLO version) can be swapped in without reworking the whole system.
NFR8	**Data Quality / Annotation Integrity**	The system should rely on consistent, correctly labeled bounding-box annotations, and should flag or handle known annotation inconsistencies (e.g., full-image boxes for defect-free samples) rather than silently degrading performance.

2.5 Business Model

The core value of FabDetector is replacing or augmenting manual visual inspection on the factory floor with a consistent, camera-driven defect detector, so this section sketches how the system could generate revenue if it were taken beyond a coursework prototype.

2.5.1 Per-Line Subscription (Primary Model)

The most direct path to revenue is a subscription charged per inspection station or per production line rather than per garment or per user. A factory would install a camera at a quality-control checkpoint, run the trained YOLOv8s model locally or on a small edge device, and pay a monthly fee that scales with the number of lines covered. A

limited free tier (for example, one line and a capped image volume) would let smaller factories or individual supervisors trial the system before committing, which matters in a price-sensitive market like Bangladesh's ready-made garment (RMG) sector, where most factories are small and medium enterprises.

2.5.2 Data and Model Licensing (Longer-Term)

A second, longer-term revenue idea is licensing the cleaned and annotated defect dataset and the trained detection weights, not the raw factory footage since that belongs to the originating factory, to buying houses, textile testing labs, or other research groups that want a validated starting point instead of building an annotated dataset from scratch. This is a B2B model rather than a per-factory subscription, and is closer to how a lot of the underlying "AI infrastructure" companies actually make money: they are not selling to individual end users at all, they are selling the pipeline and the data itself.

2.6 Social, Economic, and Environmental Impact

2.6.1 Social Impact

Manual fabric inspection is repetitive, close-focus work typically done for long shifts under artificial lighting; over time it contributes to eye strain and fatigue-related errors among inspectors, as discussed in Chapter 1. An automated first pass does not remove the need for people, but it changes what they spend their attention on: inspectors can focus on confirming flagged defects and handling edge cases rather than scanning every inch of every garment, which is a more sustainable workload over an eight-hour shift.

2.6.2 Economic Impact

For a factory, the direct economic case is fewer defective garments reaching packing and shipping, since export buyers in the RMG sector enforce strict quality thresholds and can reject entire shipments over recurring defects. Catching a hole, spot, or yarn fault earlier in the line is cheaper than reworking or discarding a finished garment.

2.6.3 Environmental Impact

Bangladesh's textile sector is both a major export earner and a significant consumer of water, dye, and raw fiber; a defective garment caught late in the process wastes all of the material, water, and energy already invested in weaving, dyeing, and stitching it. Catching defects earlier reduces this wasted embodied resource use. On the model side, we deliberately evaluated lightweight “s” variants of YOLO (YOLOv8s, YOLOv9s, YOLOv11s, YOLO26s) rather than their larger counterparts; smaller models need less GPU time to train and less compute to run inference, which keeps both the training footprint and the ongoing energy cost of deployment lower, a consideration we also examined more directly in our CSE407 Green Computing coursework.

2.6.4 Data Privacy and Security

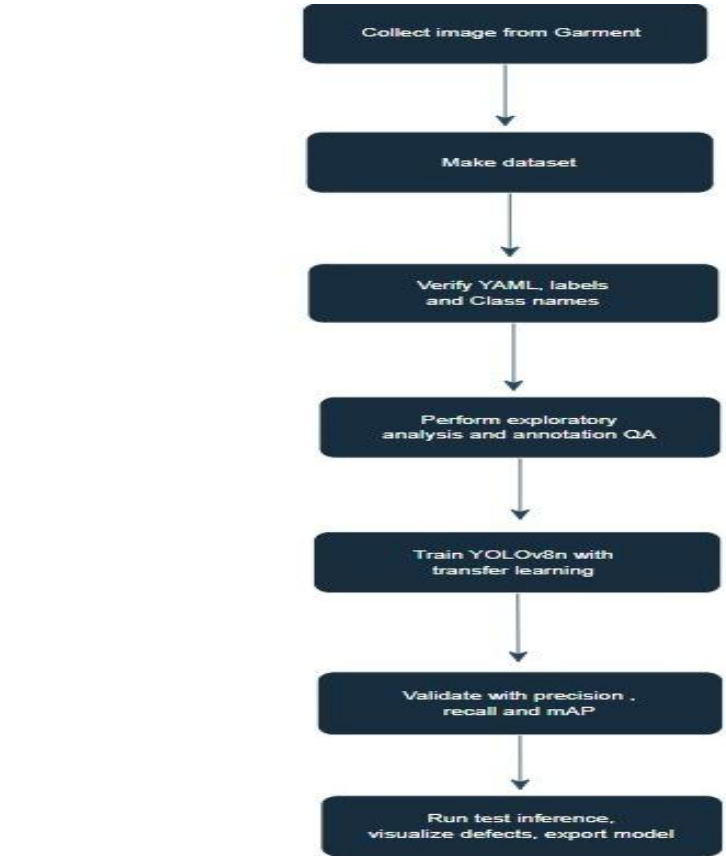
The dataset used in this project consists of fabric and garment images collected with the factory's permission at APT Garments, Gazipur; it does not target or attempt to identify individual workers. In a real deployment, camera placement and framing would need to be controlled so that inspection cameras capture fabric rather than identifiable faces, and any footage retained for retraining would need to be stored securely and access-controlled, consistent with the traceability and maintainability requirements already listed in Section 2.4.2.

Chapter 3

Methodology

3.1 Methodology

This study follows a machine learning approach to identify the shape of a garment. First, a dataset is prepared by combining images of different textile types. Data preprocessing, such as resizing, normalization, and augmentation, is then performed. Then, a suitable model for image classification is selected, which can use YOLOv8s, YOLOv9s , YOLOv11s , YOLOv26s and Feature Transfer Learning methods. The selected model was trained on the training dataset and its performance was evaluated using the validation dataset. Finally, the performance of the model is verified using the test dataset and the results are evaluated using metrics such as accuracy, precision, recall, etc. Figure 1 illustrates the overall pipeline from data collection to deployment.



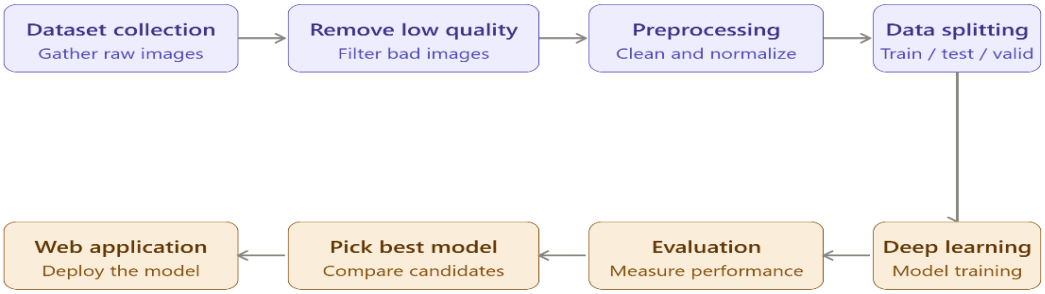


Figure 1. Workflow of the proposed fabric defect detection system.

The diagram represents a structured methodology for developing a deep learning model. Here's a breakdown of the steps:

Dataset Collection: The first step is gathering a large and diverse dataset, which serves as the foundation for training the deep learning model. Depending on the task, the dataset may include various types of data, such as images, text, or sensor data. In our project, we collected the raw data manually from a sweater factory located in **Gazipur.** We visited the factory several times to capture and gather sufficient data under different working conditions. These repeated visits ensured that the dataset was diverse, representative, and of adequate size for effectively training and evaluating the deep learning model.

Remove Low-Quality Images: Next, the collected dataset is filtered to remove low-quality images or data points that may negatively impact the performance of the model. This ensures that only high-quality data is used for further processing.

Preprocessing Dataset: The data is then preprocessed, which involves cleaning, normalizing, and transforming the data to prepare it for training. This step could include techniques like resizing images, data augmentation, or feature extraction.

Data Splitting: The preprocessed dataset is split into three parts: training, testing, and validation sets. The training set is used to teach the model, the testing set is for evaluating the model's performance after training, and the validation set helps in tuning the model's hyperparameters during the development process.

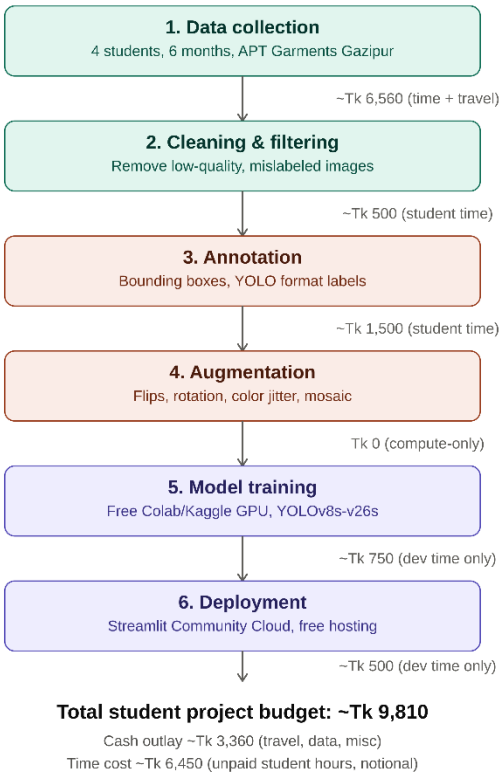
Pick the Best Model: Once the model is trained, multiple models may be compared, and the best-performing one is selected based on evaluation metrics like accuracy, precision, or recall.

Evaluation: The selected model is then evaluated on the test data to assess its generalization ability and performance on unseen data. This helps to understand the model's effectiveness.

Web Application: Finally, the trained and evaluated model is integrated into a web application, making it accessible for users to interact with and use the model's predictions in real time. Our web application is designed to detect defects from both uploaded images and video streams. Users can upload **several image** for defect analysis or process **video f**ootage to detect defects continuously in real time. This dual functionality makes the system more practical and suitable for real-world industrial quality inspection.

3.2 Data Collection and Development Cost Analysis

This section presents the end-to-end pipeline used to build FabDetector, from field data collection at APT Garments, Gazipur, through model training and deployment, with



the total project cost constrained to under Tk 10,000. This reflects a minimal, student--run project scope: a biweekly (rather than weekly) visit schedule, a low notional hourly rate, local transport, and free-tier compute and hosting throughout.

Cash outlay vs. time cost

Cost type	Estimate
Cash outlay (transport, mobile data, misc.)	~Tk 3,360
Time cost (unpaid student hours, notional)	~Tk 6,450
Total student project budget	**~Tk 9,810**

Of the roughly Tk 9,810 total project budget, only about Tk 3,360 (34%) is actual cash spent, mainly on local transport and mobile data during field visits. The remaining Tk 6,450 (66%) is the notional value of unpaid student time, included to reflect effort rather than money that actually changed hands.

3.3 Commercial Deployment Cost Analysis (Bangladesh Market)

This section estimates the cost of taking FabDetector from a working prototype to a lean, real-world commercial deployment for a Bangladeshi garment-industry client. All figures below are indicative planning assumptions for the Bangladesh market as of 2026, not vendor quotes, and should be adjusted against actual quotations before use in a real budget.

Assumptions used

Assumption	Value used
Build & launch timeline	~2 months (development to live deployment)
Development team	1 freelance CV/ML developer (part-time), 1 freelance full-stack developer (part-time),
Field data collection	1 paid technician, 5 factory visits
Data annotation	Outsourced to 1 paid freelance annotator, ~3 weeks

Model training compute	Paid cloud GPU, ~20 GPU-hours (light hyperparameter search, reusing prototype baseline)
Hosting & infrastructure	Low-cost shared VPS / lightweight cloud hosting
Post-launch operation	Minimal part-time maintenance and support, billed monthly

Development & Labor Cost (one-time build phase)

Role	Team size	Rate	Subtotal
Computer Vision / ML Developer (model training, tuning, API) — freelance	1	Tk 35,000/month × 2 mo	Tk 70,000
Full-Stack Web Developer (production app, backend, database, auth) — freelance	1	Tk 30,000/month × 1.5 mo	Tk 45,000
UI/UX design (folded into developer scope)	—	Flat	Tk 8,000
QA & coordination (folded into developer scope)	—	Flat	Tk 7,000
Field Data Collection Technician (paid, 5 visits)	1	Tk 700/visit	Tk 3,500
Data Annotation Worker (bounding-box labeling)	1	Tk 8,000/month × 1 mo	Tk 8,000
Total development & labor cost	**—**	**—**	**Tk 1,41,500**

Compute, Domain & One-Time Infrastructure Cost

Item	Estimate
Paid cloud GPU training (~20 GPU-hours × ~Tk 120/hr)	~Tk 2,400
Domain name registration (1st year)	~Tk 1,200
SSL certificate & DNS setup	Tk 0 (free via Let's Encrypt/Cloudflare)
Total one-time infrastructure cost	**~Tk 3,600**

Total One-Time Launch Cost

Cost category	Estimate
Development & labor (team, field work, annotation)	Tk 1,41,500
Compute, domain & setup	Tk 3,600
Total one-time launch cost	**~Tk 1,45,000 (~Tk 1.45 lakh)**

Recurring Monthly Operating Cost (post-launch)

Item	Estimate
Shared VPS / lightweight cloud hosting	~Tk 1,500/month
Object storage, database & backups	~Tk 300/month
Monitoring / misc. cloud services	~Tk 300/month
Part-time maintenance developer (bug fixes, retraining support)	~Tk 8,000/month
Part-time client support / onboarding	~Tk 4,000/month
Total recurring cost	**~Tk 14,100/month (~Tk 1,69,200/year)**

Estimated First-Year Total Cost of Ownership

Combining the one-time launch cost with twelve months of recurring operating cost gives an approximate first-year total cost of ownership for a lean commercial rollout of FabDetector:

Cost component	Estimate
One-time launch cost	~Tk 1,45,000
Recurring cost × 12 months	~Tk 1,69,200
Estimated first-year total cost	**~Tk 3,14,000 (~Tk 3.14 lakh)**

Labor remains the dominant cost driver even in this lean setup: the two freelance developers, technician, and annotator together account for about Tk 1,33,500 of the Tk 1,41,500 development cost, while one-time infrastructure adds only about Tk 3,600.

Chapter 4

Design and Implementation

4.1 Dataset Preparation

The dataset used in this study consists of labeled images belonging to 5 output classes with total 2416 pictures, each representing a distinct category required for the classification task. The images were originally available with varying resolutions of approximately 640×640 pixels, which were resized to 224×224 pixels to maintain consistency and ensure compatibility with standard deep learning architectures.

A total of 2416 images were allocated for training, while 1416 images were reserved for validation. An additional 480 images were used exclusively for evaluating the model during testing. To ensure proper generalization and reduce model overfitting, a validation split of 20% was applied to the training set. The dataset was then processed and loaded using TensorFlow's ImageDataGenerator, which allowed dynamic preprocessing and augmentation.

The implementation in this project utilizes a garment defect detection dataset originally organized in YOLO format, which allows the model to learn both the category and spatial location of defects through annotated bounding boxes. The original dataset consists of five classes: defect_free, hole, knot_slub, spot, and yarn_defect, representing common types of textile quality issues.

For the purpose of this project, the dataset is transformed into a binary classification problem to simplify the detection task. The classes are redefined as:

- Class 0: defect_free

- Class 1: defect (combining hole, knot_slub, spot and yarn_defect)

For effective model training and evaluation, the dataset is divided into three subsets:

- Training set: 1614 images
- Validation set: 480 images
- Test set: 322 images

This split ensures that the model can learn meaningful patterns from the training data, while the validation set is used for tuning and monitoring performance, and the test set provides an unbiased evaluation of the final model.

In total, the dataset contains approximately 3708 annotated defect instances, including both defect and defect-free samples.

However, the dataset exhibits class imbalance, where certain defect categories such as spot and defect_free are more frequent, while others like tear are relatively rare. After conversion to binary classes, this imbalance still affects the distribution between defect and defect-free samples. As a result, the model may become biased toward more frequent patterns, potentially reducing its ability to detect less common or subtle defects.

Additionally, defect-free images are annotated using full-image bounding boxes, which is not a standard approach in object detection tasks. This can influence how the model learns background features and may impact its ability to clearly distinguish between defective and non-defective regions.

The dataset used in this project is a garment defect detection dataset structured in YOLO format, where each image is paired with a corresponding annotation file containing bounding box information. These annotations allow the model to learn both the category of defects and their spatial locations within the fabric images.

The dataset consists of five classes representing common garment conditions:

- defect_free – normal fabric with no visible defects
- hole – visible openings or gaps in the fabric
- knot_slub – irregular thick yarn or knots

- spot – stains or discolorations
- yarn_defect – thin or broken yarn structures

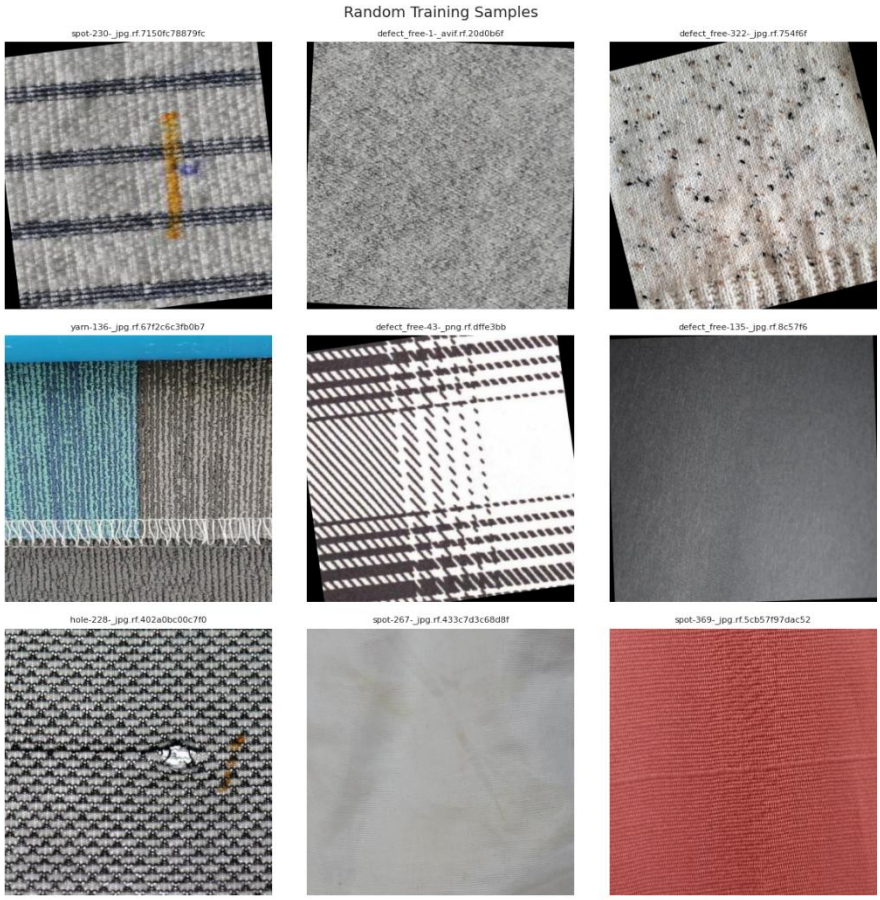


Figure 2. Some samples of dataset.

To improve robustness and enable the model to learn discriminative variations, several data augmentation techniques were applied, including:

- Image rescaling to normalize pixel intensity values between 0 and 1
- Random rotation, width and height shifting
- Shearing and zooming
- Horizontal flipping

These augmentations increased the effective dataset size and improved the model's ability to generalize to unseen samples. The final dataset pipeline consisted of three separate generators:

1. Training Generator supplies augmented images for model learning
2. Validation Generator monitors generalization performance during training
3. Test Generator evaluates model performance without augmentation

A sample of randomly selected training images was visualized to verify correct labeling and class balancing. This ensured that the dataset was properly structured and suitable for training the classification model.

4.2 Preprocessing

A comprehensive preprocessing strategy was implemented to prepare the dataset for training using YOLO. The models selected for this study include YOLOv8s, YOLOv9s , YOLOv11s and YOLOv26s**,** all of which require consistent input dimensions and normalized pixel values for optimal performance. Proper preprocessing ensured uniformity across samples and improved overall model learning capability.

4.2.1 Input Image Size

Uniformity: All input images were resized to 640x640 pixels. This uniform size is a standard practice for YOLO models and is configured via the imgsz parameter during training (imgsz = 640).

Rationale:

Computational Efficiency: Using a consistent input size simplifies the model's architecture and reduces the computational overhead during training and inference. Larger images require more memory and processing power, while smaller images might lose crucial details.

Batch Processing: Deep learning models often process images in batches. A uniform size allows for efficient batching, as all images in a batch will have the same dimensions.

Feature Learning: While the model is trained to be somewhat scale-invariant, having a standardized input size provides a consistent context for the convolutional layers to extract features related to fabric defects.

4.2.2 Data Augmentation

To improve model generalization, reduce overfitting, and simulate real-world variability**,** data augmentation was applied during training using TensorFlow's ImageDataGenerator. Augmentation generates new samples from existing images by introducing random transformations, thus increasing dataset diversity without requiring additional manual data collection.

Table 2. The augmentation strategies used in this research are summarized.

Augmentation Method	**Significance**
Rotation	Helps model recognize rotated patterns
Width & Height Shift	Introduces translational variation
Shear Transformation	Adds geometric distortion tolerance
Random Zoom	Improves scale adaptability
Horizontal Flip	Increases diversity with mirrored images
Pixel Scaling (1/255)	Standardizes illumination & contrast

This augmentation process enabled the deep learning models to learn more robust and discriminative features.

4.2.3 Normalization

While explicit pixel value normalization (e.g., scaling pixel values from [0, 255] to [0, 1]) is a common preprocessing step, it is often handled internally by the YOLO framework or the underlying deep learning libraries (like PyTorch or TensorFlow) when loading images. Typically, image data is converted to float32 and normalized to the [0, 1] range automatically before being fed into the neural network.

4.2.4 Albumentations Integration

During the offline augmentation phase using albumentations, while a dedicated Resize transform wasn't explicitly in the pipeline (as imgsz handles the final input size for YOLO training), the augmentations themselves operate on the image, and the output is re-sized if necessary. Albumentations ensures that bounding box coordinates are correctly adjusted in conjunction with any image transformations (like shifts, scales, or rotations) to maintain accurate annotations.

4.3 Models

To develop a robust image classification system capable of categorizing fabric images into 5 distinct classes, multiple deep learning architectures were implemented and compared. Specifically, three state-of-the-art YOLO (You Only Look Once) models were utilized as feature extractors:

1. YOLOv8s
2. YOLOv9s
3. YOLOv11s
4. YOLO26s

These models were selected due to their proven performance in fabric defect image analysis and their efficiency in extracting discriminative features from visual data.

The YOLO (You Only Look Once) architecture, specifically the YOLOv8s, YOLOv11s variant used in this project, represents a paradigm shift in object detection. Unlike traditional two-stage detectors that first propose regions and then classify them, YOLO operates as a single-stage detector. It divides the input image into a grid, and for each grid cell, it simultaneously predicts bounding boxes, confidence scores (indicating the likelihood of an object being present within that box), and class probabilities. This end-to-end approach allows YOLO to achieve remarkably fast inference speeds, making it highly suitable for real-time applications such as on-the-fly fabric defect detection.

The core of the YOLO architecture consists of a powerful backbone network (often a variant of CSPDarknet or similar efficient CNN structures) for feature extraction, followed by a neck that aggregates features at different scales, and finally, a head that performs the actual detection. The 's' (small) designation indicates a more compact version of the YOLO model, optimized for efficiency with fewer parameters and computational demands while still maintaining strong performance. The architectural design enables the model to learn global context from the entire image, leading to fewer background errors and a more unified understanding of objects within the scene, a crucial advantage for accurately identifying subtle fabric defects.

Generalized Architecture Framework:

Table 2. Architectural Comparison of the Evaluated YOLO Models.

Architecture Component	YOLOv8s	YOLOv9s	YOLOv11s	YOLO26s
Release Year	2023	2024	2024	2025
Framework	Ultralytics	WongKinYiu	Ultralytics	Ultralytics

Architecture Component	YOLOv8s	YOLOv9s	YOLOv11s	YOLO26s
Input Resolution	640 × 640	640 × 640	640 × 640	640 × 640
Backbone	CSP-based backbone with C2f blocks	GELAN (Generalized Efficient Layer Aggregation Network)	Enhanced C3k2/C2PSA backbone	Optimized lightweight CSP-based backbone
Neck	PAN-FPN	PAN-FPN with GELAN aggregation	Improved PAN-FPN with attention optimization	Optimized PAN-FPN
Detection Head	Anchor-free, decoupled head	Anchor-free, decoupled head	Anchor-free, decoupled head	Anchor-free, decoupled head
Bounding Box Regression	Distribution Focal Loss (DFL)	DFL	DFL	DFL
Feature Pyramid Levels	P3, P4, P5	P3, P4, P5	P3, P4, P5	P3, P4, P5
Multi-scale Detection	Yes	Yes	Yes	Yes
Anchor Mechanism	Anchor-free	Anchor-free	Anchor-free	Anchor-free
Primary Optimization	Speed–Accuracy Balance	Improved Feature Learning	Better Accuracy and Efficiency	Lightweight Optimization
Main Strength	High detection accuracy with low computational cost	Better gradient propagation and feature aggregation	Improved feature extraction and localization	Faster convergence with competitive accuracy
Typical Industrial Application	Real-time defect detection	Complex object detection	High-accuracy industrial inspection	Resource-constrained edge deployment

4.4 Model Evaluation

4.4.1 YOLOv8s

The YOLOv8s model was evaluated for automated garment defect detection using five classes: defect_free, hole, knot_slub, spot, and yarn. The training and validation curves demonstrate stable convergence throughout the training process, indicating that the model successfully learned representative features from the garment defect dataset.

Training Performance

The training and validation loss curves consistently decreased over the training epochs, demonstrating effective optimization and convergence of the network. Although the validation loss exhibited a sharp fluctuation during the initial epochs, it quickly stabilized and closely followed the training loss. The small gap between the training and validation losses at the end of training suggests that the model experienced minimal overfitting and generalized well to unseen data.



Figure 3. YOLOv8s Training and Validation Loss and Validation Accuracy.

The bounding-box loss, classification loss, and Distribution Focal Loss (DFL) all showed continuous reductions during training. Simultaneously, precision, recall, and

mean Average Precision (mAP) increased steadily before reaching a plateau, indicating that the learning process had converged successfully.

Detection Performance

The validation performance shows that the model achieved approximately 0.99 mAP@50 and 0.76 mAP@50–95, indicating excellent object localization under the standard IoU threshold and strong overall detection capability across multiple IoU thresholds. The gradual improvement of these metrics throughout training demonstrates the robustness and stability of the YOLOv8s architecture for garment defect detection.



Figure 4. YOLOv8s Per-class metrics.

Per-Class Performance

The per-class mAP@50 results reveal differences in detection difficulty among the garment defect categories.

Table 3. YOLOv8s Per-Class Performance values.

Class	mAP@50
Defect-free	**0.993**
Hole	**0.719**
Knot Slub	**0.693**
Spot	**0.732**
Yarn	**0.588**

The defect-free category achieved the highest detection accuracy (0.993), indicating that the model can reliably distinguish normal fabric samples. Among the defect categories, spot and hole achieved relatively high detection performance, whereas knot slub showed slightly lower accuracy. The yarn defect obtained the lowest mAP (0.588), suggesting that this defect type is more challenging because of its irregular appearance, thin structure, and visual similarity to surrounding fabric textures.

Confusion Matrix Analysis

The confusion matrix further confirms the effectiveness of YOLOv8s. Most samples are correctly classified along the diagonal, demonstrating high class discrimination capability. Very few misclassifications occur among the defect categories, indicating that the learned feature representations are highly discriminative.

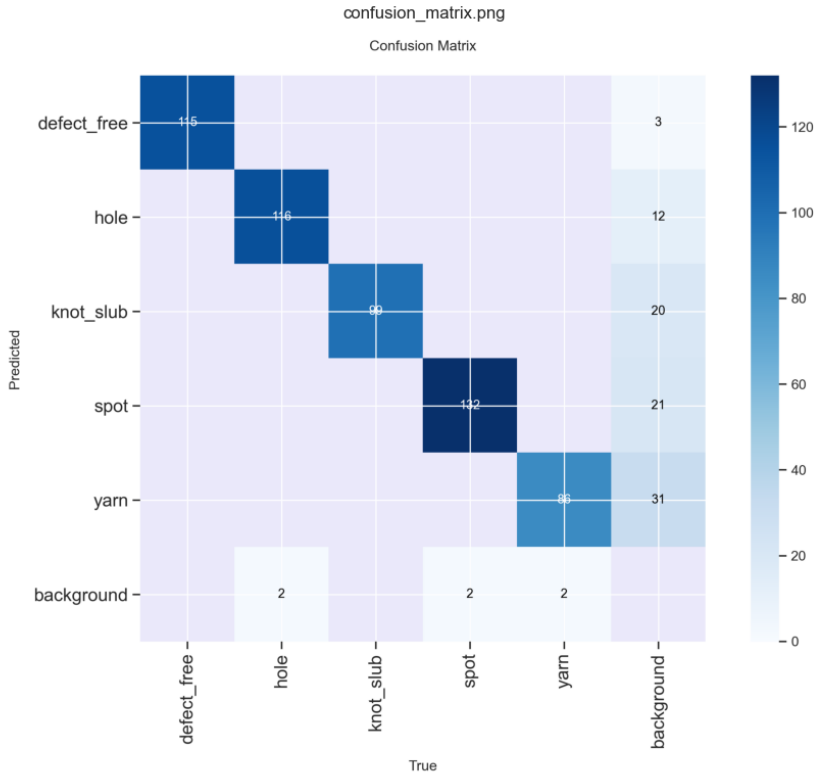


Figure 5. YOLOv8s Confusion Matrix.

The largest source of prediction errors is associated with the background class, where several defect instances are either missed or partially detected. This behavior is common in object detection tasks involving small or low-contrast defects, particularly when defects occupy only a limited portion of the fabric surface. Nevertheless, confusion between the actual defect classes remains relatively low, indicating that once a defect is detected, the model usually identifies its correct category.

Overall Assessment

The experimental results demonstrate that YOLOv8s provides an excellent balance between detection accuracy and computational efficiency for automated garment defect

inspection. The model exhibits stable convergence, high localization accuracy, and strong classification performance across most defect categories. Although detection performance for yarn defects is comparatively lower, the overall results indicate that YOLOv8s is highly suitable for real-time industrial garment quality inspection, where both speed and accuracy are critical. Future improvements may focus on enhancing the detection of fine-textured defects through additional data augmentation, higher-resolution training images, or attention-based feature enhancement modules.

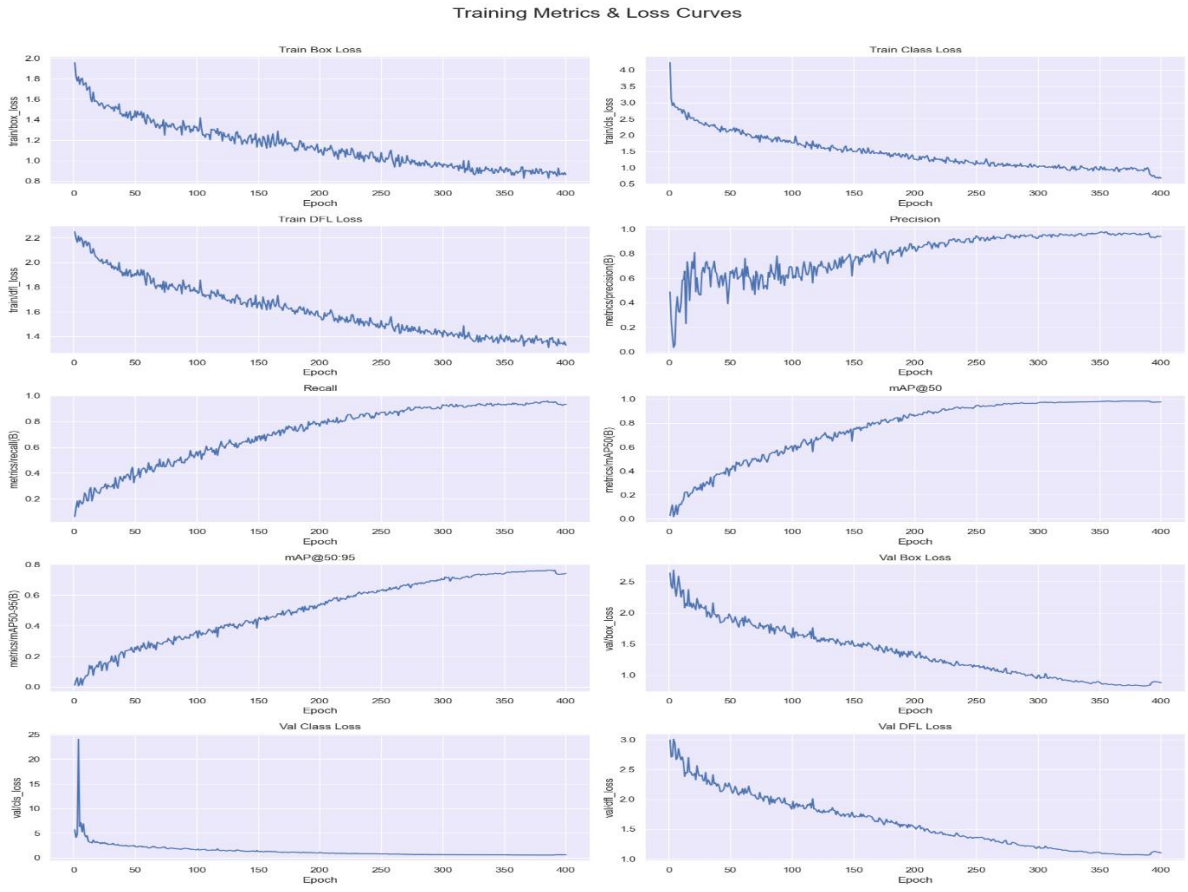


Figure 6. YOLOv8s Training metrics and loss curves.

4.4.2 YOLOv9s

The YOLOv9s model was evaluated on the garment defect dataset containing five classes: defect_free, hole, knot_slub, spot, and yarn. The evaluation was conducted by analyzing the training and validation loss curves, detection metrics, per-class performance, and confusion matrix to assess the model's learning capability and generalization performance.

Training Performance

The training and validation loss curves demonstrate a stable learning process throughout the training period. Both losses decrease consistently as the number of epochs increases, indicating effective optimization of the network parameters. The validation loss follows a trend similar to the training loss with only minor fluctuations, suggesting that the model generalizes well without exhibiting significant overfitting.

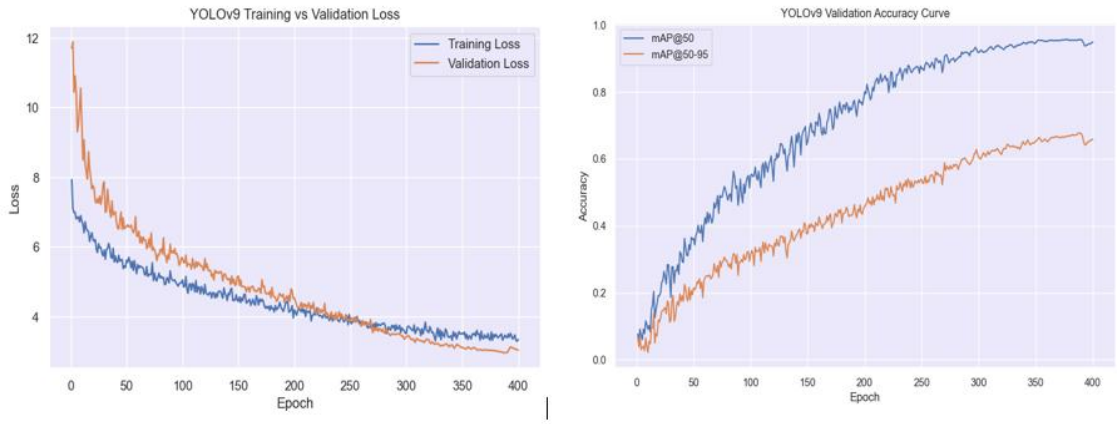


Figure 7. YOLOv9s Training and Validation Loss and Validation Accuracy.

The individual loss components, including bounding-box loss, classification loss, and Distribution Focal Loss (DFL), all decrease steadily during training. Meanwhile, the evaluation metrics precision, recall, mAP@50, and mAP@50–95 show continuous

improvement before stabilizing toward the later epochs, indicating successful convergence of the YOLOv9s architecture.

Detection Performance

The validation metrics indicate that YOLOv9s achieved approximately 0.94 mAP@50 and 0.68 mAP@50–95 after convergence. These results demonstrate that the model is capable of accurately localizing and classifying garment defects under different IoU thresholds.

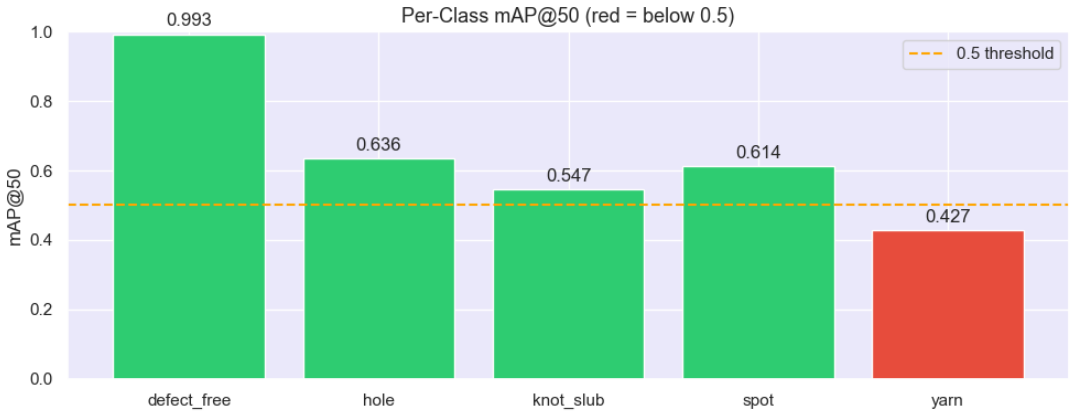


Figure 8. YOLOv9s Per-class metrics.

Per-Class Performance

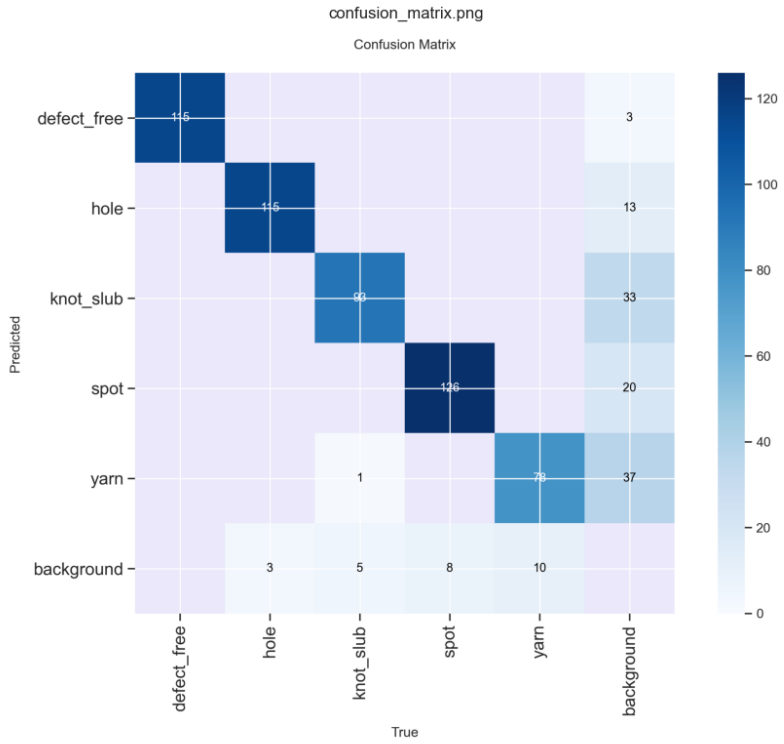
The per-class mAP@50 values are summarized below.

Table 4. YOLOv9s Per-Class Performance values.

Class	mAP@50
Defect-free	**0.993**
Hole	**0.636**
Knot Slub	**0.547**
Spot	**0.614**
Yarn	**0.427**

The defect-free category achieved the highest detection accuracy with an mAP@50 of 0.993, demonstrating excellent recognition of normal fabric images. Among the defect categories, hole and spot defects obtained moderate detection performance, while knot slub achieved a slightly lower score. The yarn class recorded the lowest mAP@50 (0.427), falling below the 0.5 threshold. This comparatively lower performance indicates that yarn defects remain the most challenging category due to their thin structure, irregular patterns, and visual similarity to surrounding fabric textures.

Confusion Matrix Analysis



The confusion matrix indicates that the majority of garment samples are correctly classified, as evidenced by the strong concentration of predictions along the main diagonal.

Figure 9. YOLOv9s Confusion Matrix.

However, greater confusion is observed for the knot slub and yarn classes. Several instances of these defects are incorrectly classified as background or are missed during detection, reflecting the inherent difficulty of identifying fine-textured and low-contrast defects. Despite these challenges, confusion among the major defect categories remains relatively limited, indicating that YOLOv9s effectively distinguishes different defect types once an object is successfully detected.

Overall Assessment

The experimental results demonstrate that YOLOv9s provides reliable performance for automated garment defect detection, with stable convergence, high localization capability, and consistent classification accuracy across most defect categories. The model effectively detects common defects such as hole and spot, while maintaining excellent recognition of defect-free fabric samples. Nevertheless, its performance decreases for more challenging defects, particularly yarn, which lowers the overall mean Average Precision.

Compared with the YOLOv8s model, YOLOv9s exhibits slightly lower detection performance across most defect classes, particularly in terms of mAP@50, mAP@50–95, and per-class accuracy for fine-textured defects. Although the model remains suitable for automated industrial inspection, further improvements such as enhanced feature extraction, attention mechanisms, or additional data augmentation could improve its ability to detect subtle garment defects and enhance overall detection robustness.

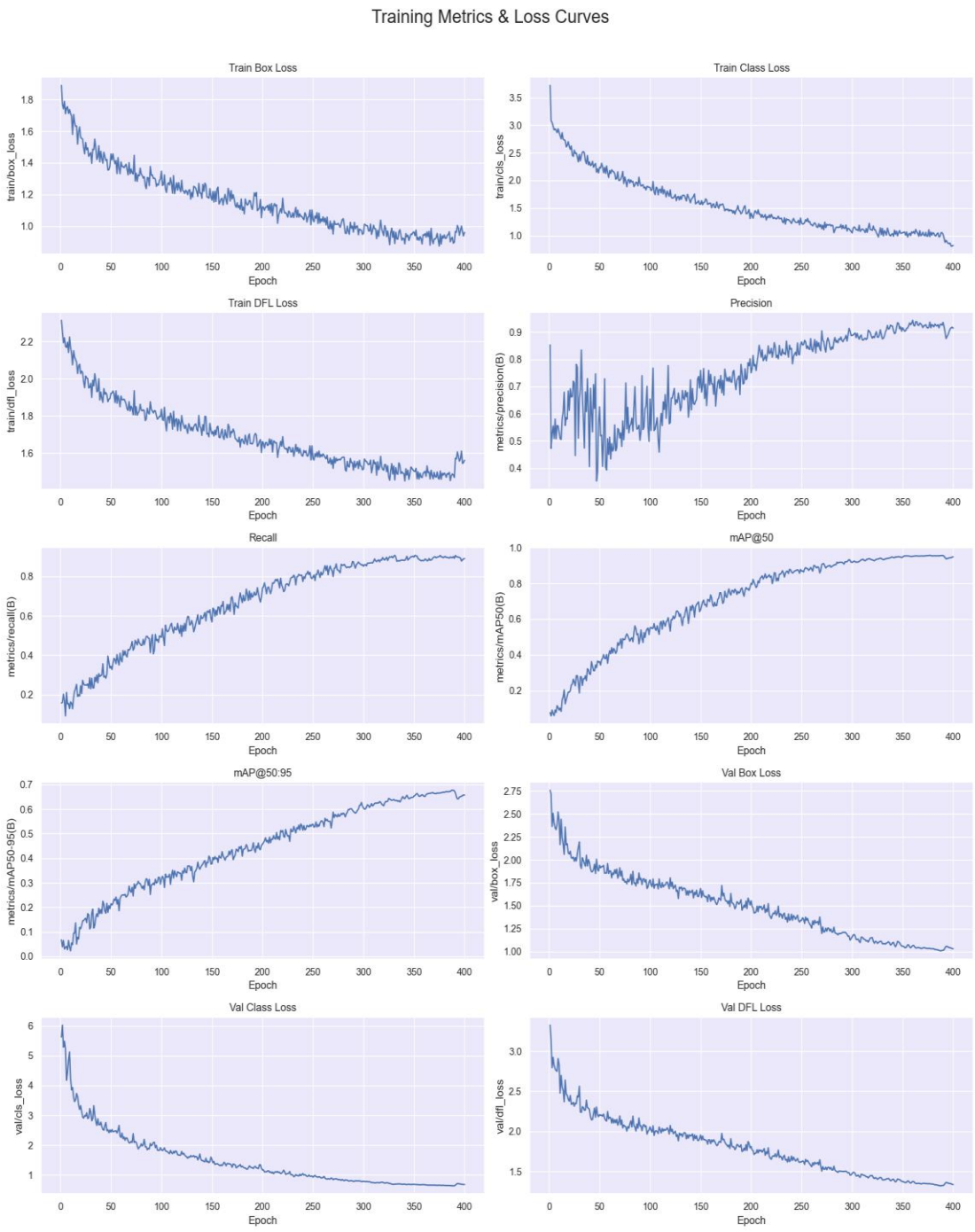


Figure 10. YOLOv9s Training metrics and loss curves.

4.4.3 YOLOv11s

The YOLOv11s model was evaluated on the garment defect dataset comprising five classes: defect_free, hole, knot_slub, spot, and yarn. The evaluation considered the model's convergence behavior, validation performance, per-class detection accuracy, and confusion matrix to determine its effectiveness for automated garment defect detection.

Training Performance

The training and validation loss curves indicate that YOLOv11s converges smoothly throughout the training process. Both the training and validation losses decrease steadily with increasing epochs, demonstrating stable optimization and effective feature learning. Although the validation loss exhibits a sharp increase during the initial epochs, it rapidly stabilizes and closely follows the training loss, indicating good generalization with minimal signs of overfitting.

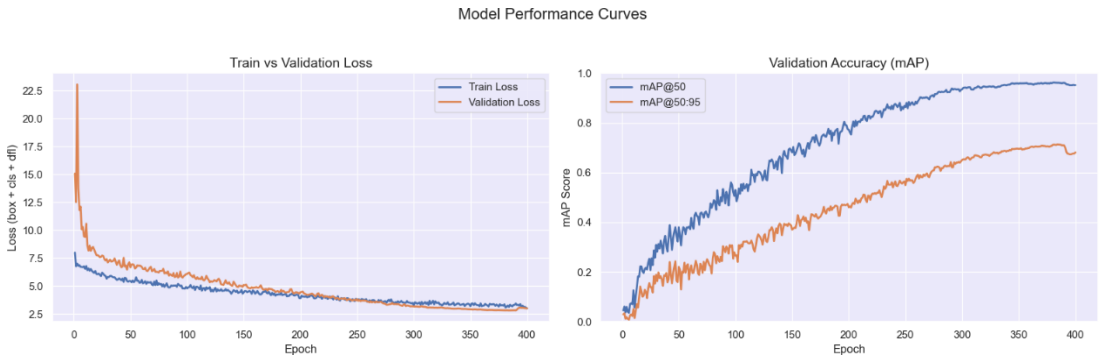


Figure 11. YOLOv11s Training and Validation Loss and Validation Accuracy.

The individual loss components, including bounding-box loss, classification loss, and Distribution Focal Loss (DFL), consistently decrease during training. Simultaneously, the evaluation metrics precision, recall, mAP@50, and mAP@50–95 increase steadily and converge toward the end of training. This behavior confirms that the model

successfully learns discriminative representations for garment defect detection while maintaining stable optimization.

Detection Performance

The validation curves show that YOLOv11s achieved approximately 0.97 mAP@50 and 0.71 mAP@50–95 after convergence. These results demonstrate strong localization and classification capabilities across different IoU thresholds. The continuous improvement in precision and recall further indicates that the model effectively balances false positives and false negatives while maintaining reliable detection accuracy.



Figure 12. YOLOv11s Per-class metrics.

Per-Class Performance

The per-class detection performance is summarized in Table X.

Table 5. YOLOv11s Per-Class Performance values.

Class	mAP@50
Defect-free	**0.986**
Hole	**0.695**
Knot Slub	**0.602**
Spot	**0.686**
Yarn	**0.435**

The defect-free class achieved the highest detection accuracy with an mAP@50 of 0.986, confirming the model's excellent ability to recognize normal fabric samples. Among the defect categories, hole and spot achieved relatively high detection performance, while knot slub showed moderate accuracy. Similar to the previous models, the yarn defect recorded the lowest mAP@50 (0.435), indicating that fine and irregular yarn defects remain difficult to detect because of their subtle visual characteristics and low contrast against the fabric background.

Confusion Matrix Analysis

The confusion matrix demonstrates that YOLOv11s correctly classifies the majority of garment samples, as reflected by the dominant values along the diagonal. The defect-free, hole, and spot categories exhibit particularly strong classification performance with minimal confusion among these classes.

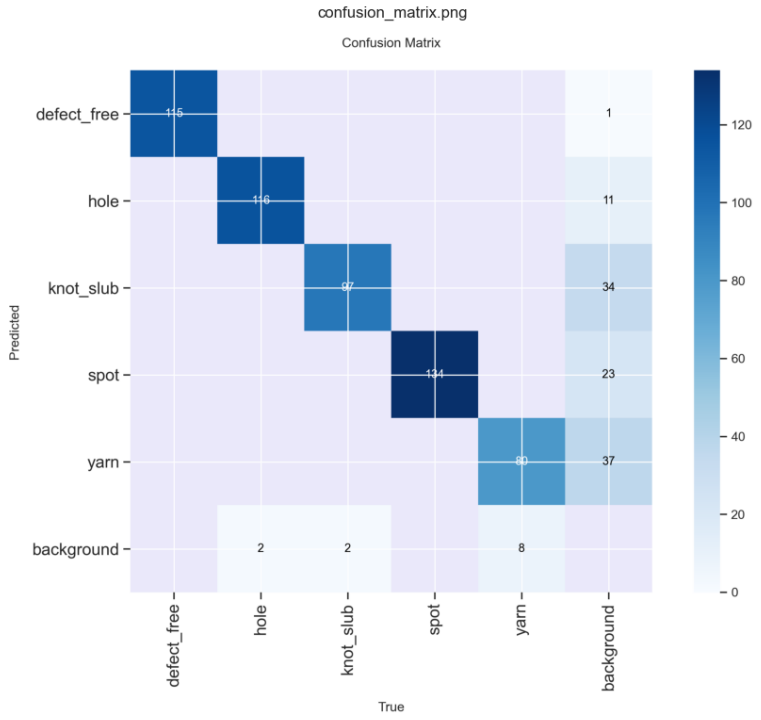


Figure 13. YOLOv11s Confusion Matrix.

Most prediction errors are associated with the background class, where several defect instances are either missed or only partially detected. The knot slub and yarn classes contribute the largest proportion of these missed detections, reflecting the increased difficulty of recognizing defects with irregular textures and small object sizes. Nevertheless, direct misclassification between different defect categories remains relatively low, suggesting that once a defect is localized, the model generally assigns it to the correct class.

Overall Assessment

The experimental results indicate that YOLOv11s is a robust and reliable detector for automated garment defect inspection. The model exhibits stable convergence, high localization accuracy, and strong classification performance across most garment defect categories. Its high mAP@50 and mAP@50–95 values demonstrate effective detection under varying IoU thresholds, making it suitable for practical industrial inspection systems.

Compared with YOLOv9s, YOLOv11s provides noticeable improvements in overall detection accuracy, particularly for the hole, knot slub, and spot classes, while maintaining comparable performance for defect-free samples. However, YOLOv8s still achieves the best overall performance among the evaluated lightweight models, especially in detecting challenging defects such as yarn, where YOLOv11s continues to experience reduced detection accuracy. These findings suggest that YOLOv11s offers an effective balance between detection performance and computational efficiency, although further enhancements such as attention mechanisms, higher-resolution feature extraction, or targeted data augmentation could further improve the detection of fine-textured garment defects.

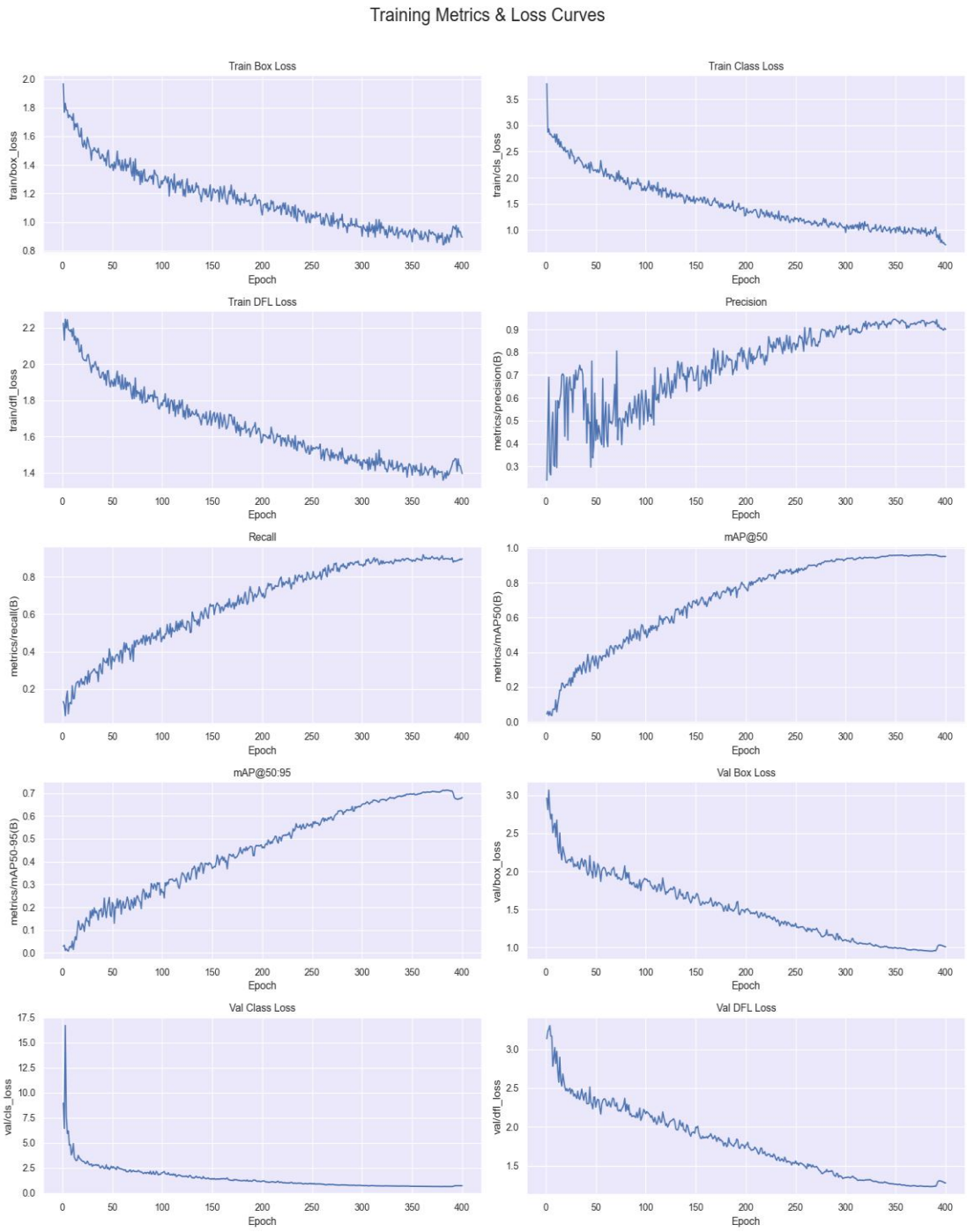


Figure 14. YOLOv11s Training metrics and loss curves.

4.4.4 YOLO26s

The YOLO26s model was evaluated on the garment defect dataset containing five classes: defect_free, hole, knot_slub, spot, and yarn. The model's performance was analyzed using the training and validation loss curves, detection metrics, per-class mean Average Precision (mAP), and confusion matrix to assess its suitability for automated garment defect detection in industrial environments.

Training Performance

The training and validation loss curves demonstrate a stable and well-converged learning process throughout the training period. Both losses decrease rapidly during the initial epochs before gradually stabilizing, indicating efficient optimization of the network parameters. The validation loss closely follows the training loss after the early training stage, suggesting good generalization capability with minimal evidence of overfitting.



Figure 15. YOLO26s Training and Validation Loss and Validation Accuracy.

The individual optimization losses including bounding-box loss, classification loss, and Distribution Focal Loss (DFL) all decrease consistently as training progresses. At the

same time, precision, recall, mAP@50, and mAP@50–95 exhibit continuous improvement before reaching stable values near the end of training. These trends confirm that YOLO26s effectively learns discriminative features while maintaining a stable optimization process.

Detection Performance

The validation curves indicate that YOLO26s achieved approximately 0.98 mAP@50 and 0.71 mAP@50–95 after convergence. These results demonstrate excellent object localization and classification performance across multiple IoU thresholds. The steady increase in precision and recall throughout training further indicates that the model effectively minimizes both false-positive and false-negative detections, making it suitable for reliable garment defect inspection.

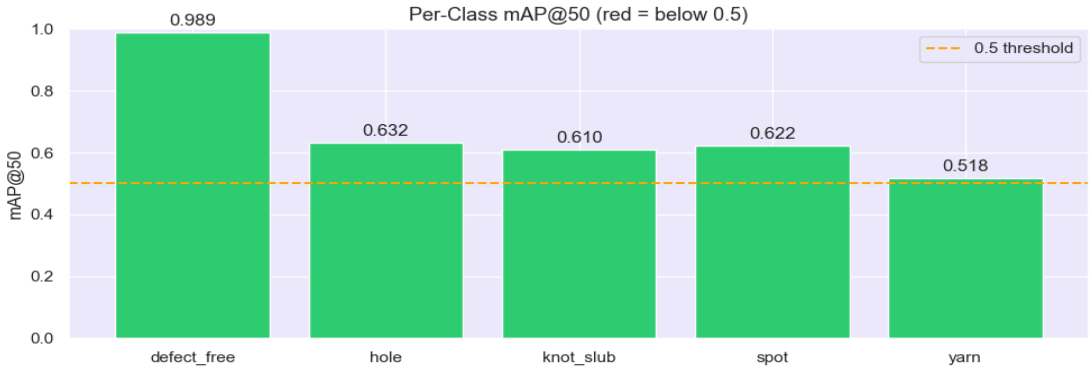


Figure 16. YOLO26s Per-class metrics.

Per-Class Performance

The per-class detection performance is summarized below.

Table 6. YOLO26s Per-Class Performance values.

Class	mAP@50
Defect-free	**0.989**
Hole	**0.632**
Knot Slub	**0.610**
Spot	**0.622**
Yarn	**0.518**

The defect-free category achieved the highest detection performance with an mAP@50 of 0.989, demonstrating the model's excellent ability to recognize normal fabric samples. Among the defect categories, hole, spot, and knot slub exhibited relatively balanced detection performance, each achieving mAP values above 0.60. Notably, the yarn defect achieved an mAP@50 of 0.518, exceeding the 0.5 threshold and outperforming both YOLOv9s and YOLOv11s for this particularly challenging defect category. This improvement indicates that YOLO26s is more effective at detecting fine and elongated defect structures.

Confusion Matrix Analysis

The confusion matrix shows that the majority of garment samples are correctly classified, with high values concentrated along the principal diagonal. The defect-free, hole, and spot categories exhibit strong classification accuracy with relatively few incorrect predictions. Misclassification among the primary defect classes remains limited, demonstrating that YOLO26s effectively learns discriminative feature representations for different garment defects.

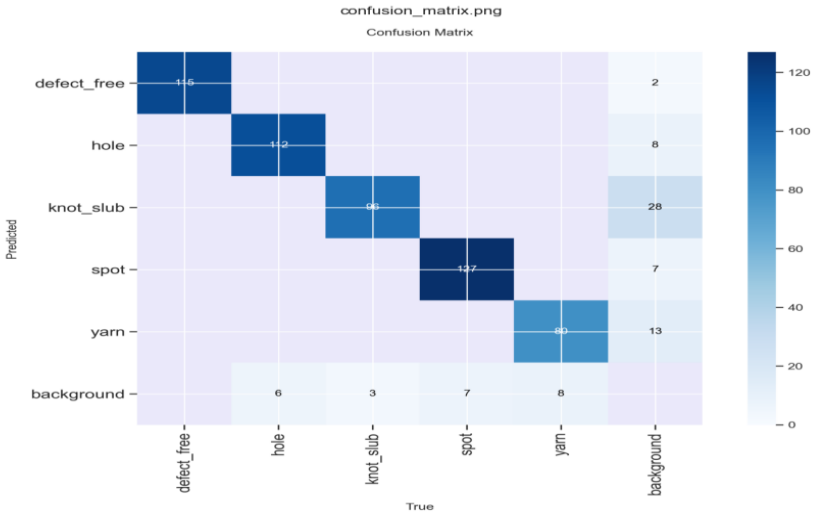


Figure 17. YOLO26s Confusion Matrix.

Most prediction errors are associated with the background class, where a small number of defects are either missed or partially localized. Similar to the previous models, knot slub and yarn remain the most challenging defect categories because of their irregular texture and low visual contrast. However, the reduced confusion involving the yarn class compared with YOLOv9s and YOLOv11s indicates that YOLO26s provides improved feature representation for these subtle defects.

Overall Assessment

The experimental results demonstrate that YOLO26s is a robust and effective model for automated garment defect detection. The model exhibits stable convergence, high localization accuracy, and consistent detection performance across all garment defect categories. The smooth reduction of training losses, together with the steady improvement of precision, recall, and mAP, confirms the reliability and robustness of the training process.

Compared with the previously evaluated lightweight models, YOLO26s achieves performance that is competitive with YOLOv11s while providing a noticeable improvement in detecting the yarn defect, which has consistently been the most difficult class across all experiments. Although YOLOv8s still delivers the highest overall per-class detection accuracy, YOLO26s offers a more balanced performance across the challenging defect categories, particularly by increasing the detection accuracy of fine-textured defects above the acceptable performance threshold. These characteristics make YOLO26s a promising candidate for real-time industrial garment inspection systems, where both high detection accuracy and robust generalization across diverse defect types are essential.

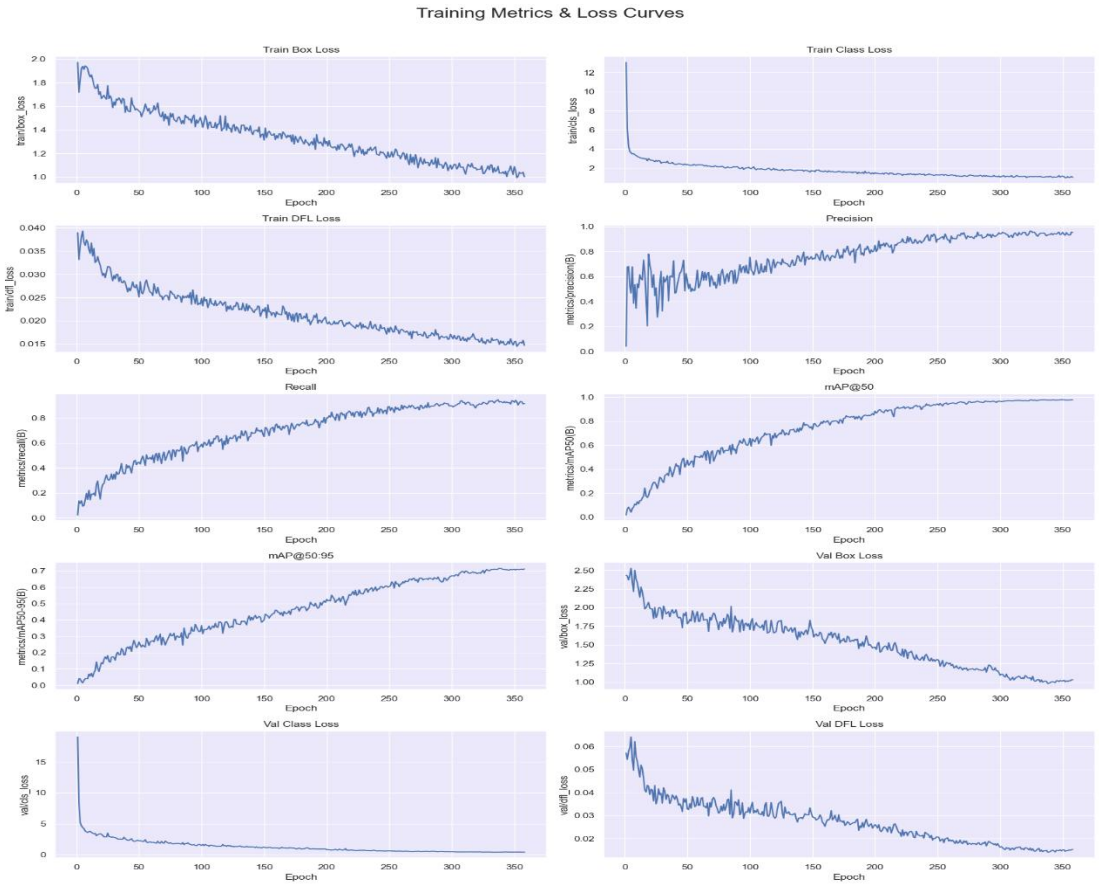


Figure 18. YOLO26s Training metrics and loss curves.

4.5 Prototype Web Application

To demonstrate the practical applicability of the proposed garment defect detection system, a prototype web application was developed using the Streamlit framework. The application provides a simple and interactive graphical user interface (GUI) that enables users to upload garment images or videos and perform automated defect detection without requiring technical expertise.

The user interface is divided into two main sections. The left sidebar contains a confidence threshold slider that allows users to adjust the detection confidence level between 0.0 and 1.0. This feature enables users to control the sensitivity of the object detector according to different inspection requirements..

The main content area allows users to upload either a garment image or a video through a file uploader. After selecting the input file and pressing the "Detect Defects" button, the application processes the input using the trained YOLO model. The detected defects are highlighted with colored bounding boxes, class labels, and confidence scores. The processed image or video is then displayed directly within the application, allowing users to visually inspect the detection results.

In addition to the visual output, the application provides a summary of the detected defect classes, the total number of detected defects, and the inference time required for processing. The prototype demonstrates the feasibility of deploying the proposed deep learning model as an intelligent quality control tool for real-time textile inspection.

Table 7: Features of the Prototype

Feature	Description
Framework	Streamlit
Model	YOLOv8s (Best Performing Model)
Input	Image (.jpg, .png, .jpeg) and Video (.mp4, .avi, .mov)
Confidence Slider	Adjustable threshold (0.0–1.0)
Detection Output	Bounding boxes, class labels, confidence scores
Result Summary	Number of detected defects by category
Performance	Real-time inference
User Interface	Web-based, interactive, and lightweight

Figure 19: Prototype web application for automated garment defect detection.

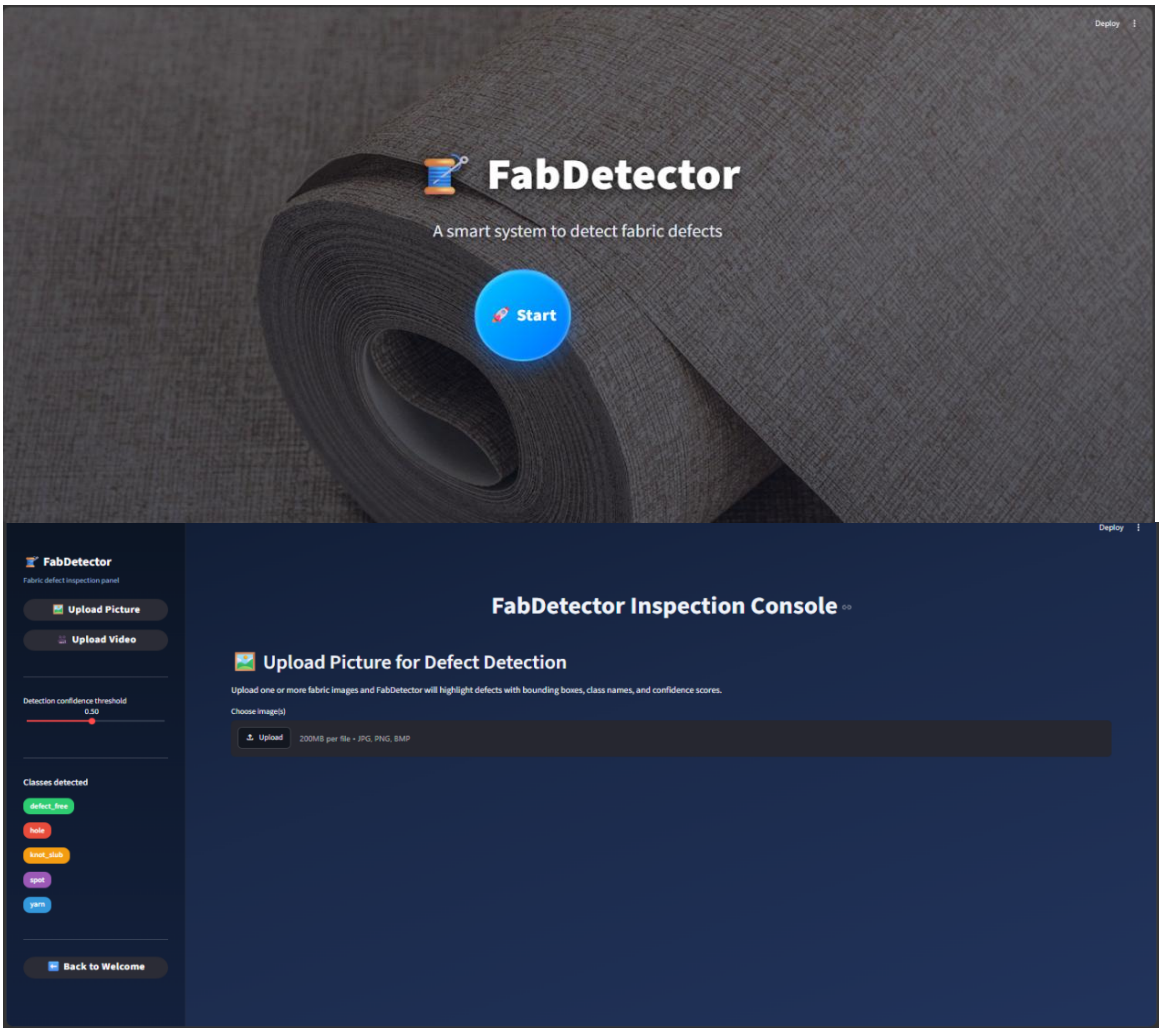
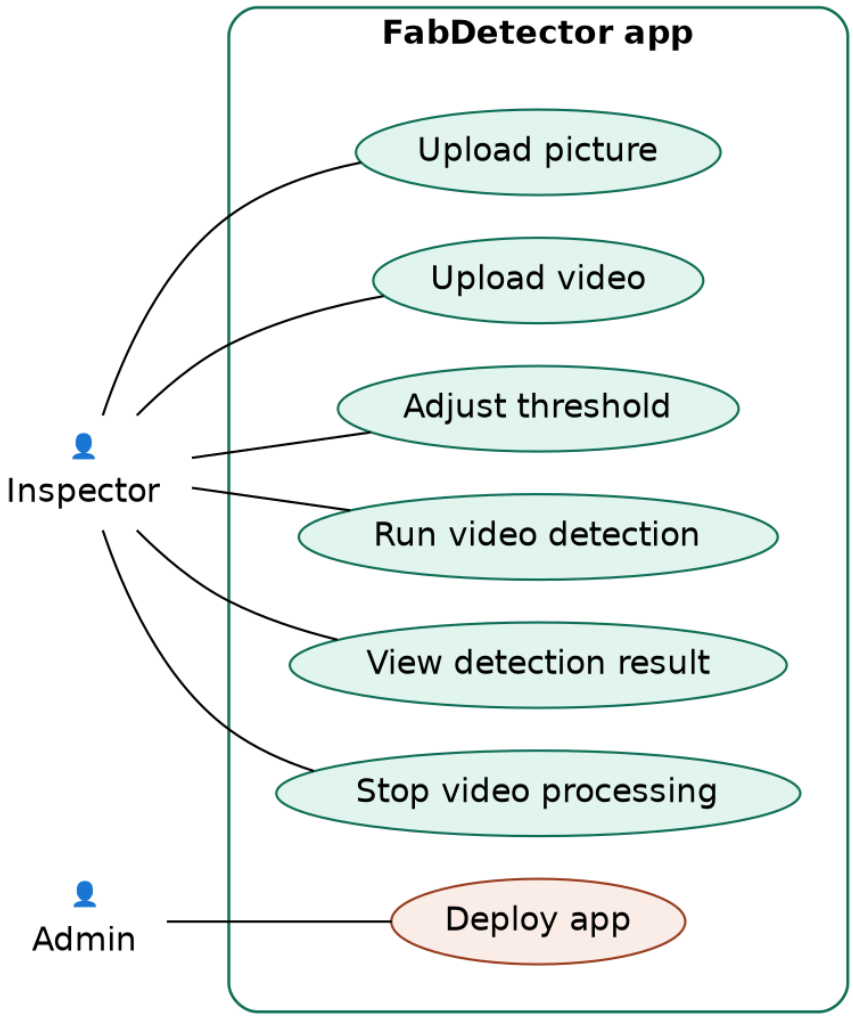


Figure 19. Prototype web application for automated fabric defect detection developed using the Streamlit framework. The left sidebar provides model configuration and confidence threshold adjustment, while the main interface allows users to upload garment images or videos and visualize the defect detection results. This prototype demonstrates how the proposed YOLO-based detection framework can be integrated into a practical, user-friendly web application for automated garment quality inspection in industrial environments.

4.6 FabDetector Application — UML Diagrams

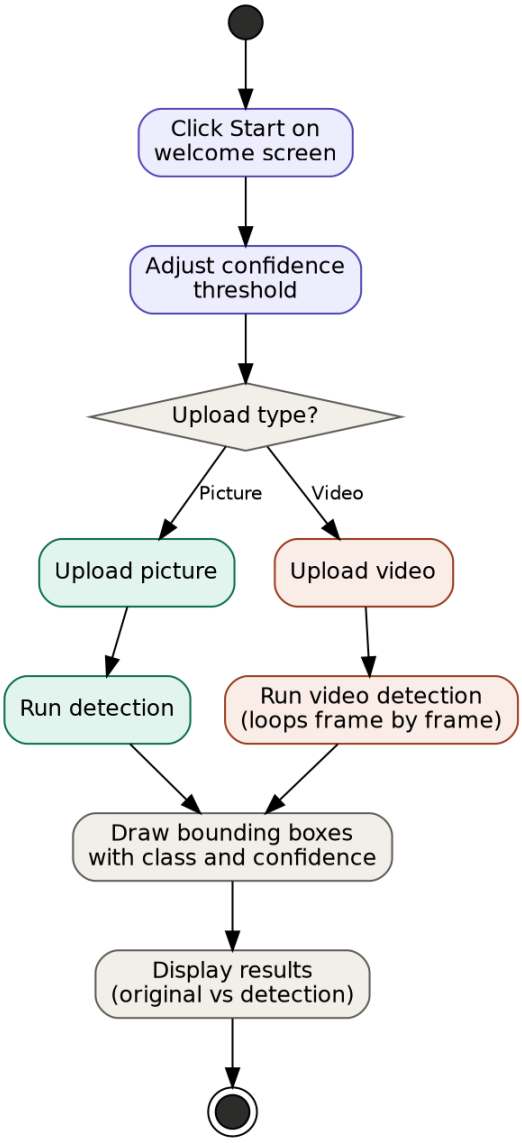
4.6.1. Use Case Diagram

The inspector uploads pictures or video, adjusts the detection confidence threshold, runs detection, views results, and can stop video processing. The admin actor is responsible for deploying the app.



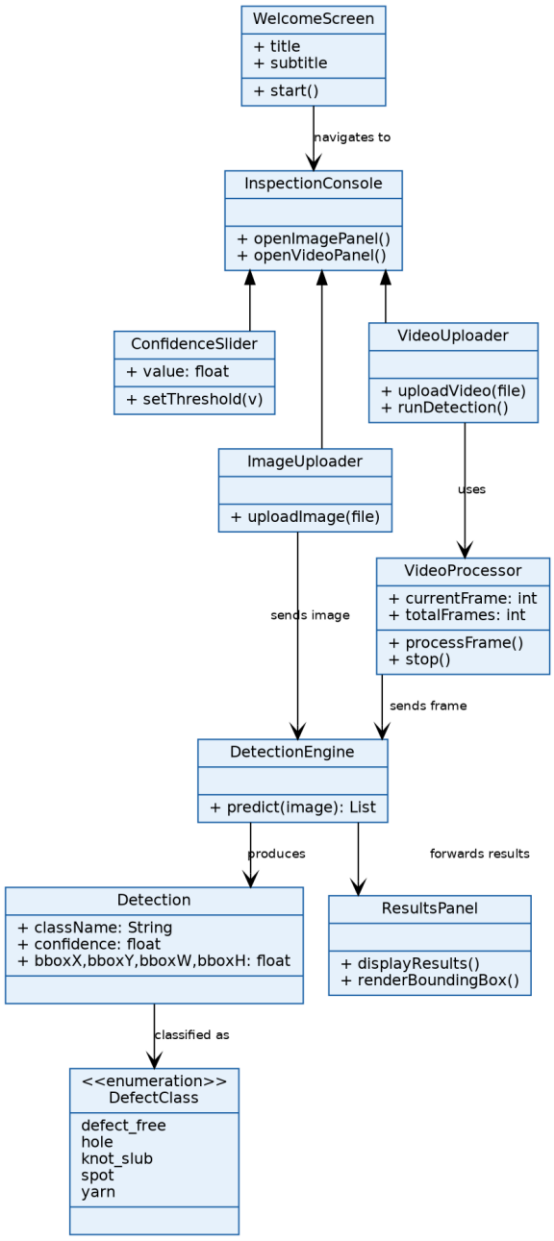
4.6.2. Activity Diagram

Runtime flow of the application: the user starts the app, sets the confidence threshold, then either uploads a picture or a video. Both paths converge on drawing bounding boxes and displaying the detection results.



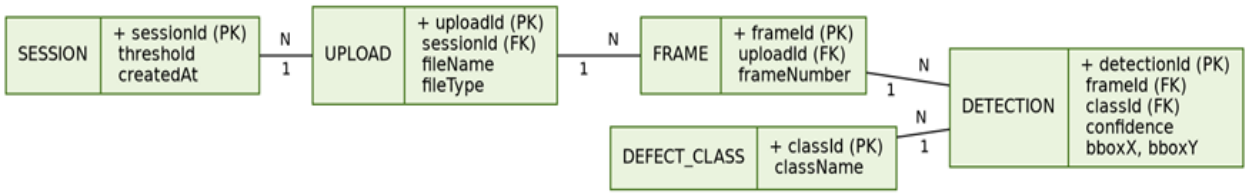
4.6.3. Class Diagram

Structural view of the application: WelcomeScreen leads to the InspectionConsole, which owns the ConfidenceSlider, ImageUploader, and VideoUploader. Uploaders feed the DetectionEngine, which produces Detection objects classified against the DefectClass enumeration and forwarded to the ResultsPanel.



4.6.4. ER Diagram

Logical data model for an inspection session: a SESSION contains UPLOADs, each UPLOAD contains FRAMEs (a single frame for images, many for video), and each FRAME yields DETECTIONs classified against DEFECT_CLASS.



Chapter 5

Results & Discussion

To evaluate the effectiveness of the proposed automated garment defect detection system, four lightweight YOLO-based object detection models, namely YOLOv8s, YOLOv9s, YOLOv11s, and YOLO26s, were trained and tested using the same dataset and experimental settings. The evaluation was performed using Precision, Recall, F1-score, mAP@50, and mAP@50:95, which are widely accepted performance metrics for object detection.

5.1 Overall Performance Comparison

Table 8 summarizes the quantitative performance of the four evaluated models.

Table 8: Performance Comparison of YOLO Models.

Model	Best Epoch	Precision (%)	Recall (%)	F1 Score (%)	mAP@50 (%)	mAP@50:95 (%)
YOLOv8s	**372**	**96.27**	**94.28**	**95.27**	**98.56**	**75.83**
YOLOv9s	377	93.71	89.86	91.75	95.73	66.69
YOLOv11s	381	93.07	89.58	91.29	96.26	71.24
YOLO26s	**349**	94.02	93.48	93.75	97.95	70.75

The experimental results demonstrate that YOLOv8s achieved the highest overall detection performance among all evaluated models. It obtained the highest Precision (96.27%), Recall (94.28%), F1-score (95.27%), mAP@50 (98.56%), and mAP@50:95 (75.83%), indicating superior localization and classification capability. Although YOLO26s required fewer training epochs (349) than the other models, it achieved competitive performance with a Precision of 94.02%, Recall of 93.48%, and mAP@50 of 97.95%. This suggests that YOLO26s converges more rapidly while maintaining strong detection performance, making it attractive for computationally constrained industrial applications. YOLOv11s demonstrated improved localization capability compared with YOLOv9s, achieving an mAP@50:95 of 71.24% compared to 66.69%. However, its Precision, Recall, and F1-score remained slightly lower than those of YOLO26s. YOLOv9s produced the lowest overall performance among the evaluated models. Although its mAP@50 remained relatively high (95.73%), the lower Recall (89.86%) and mAP@50:95 (66.69%) indicate greater difficulty in detecting challenging garment defects under stricter localization criteria.

5.2 Comparative Analysis

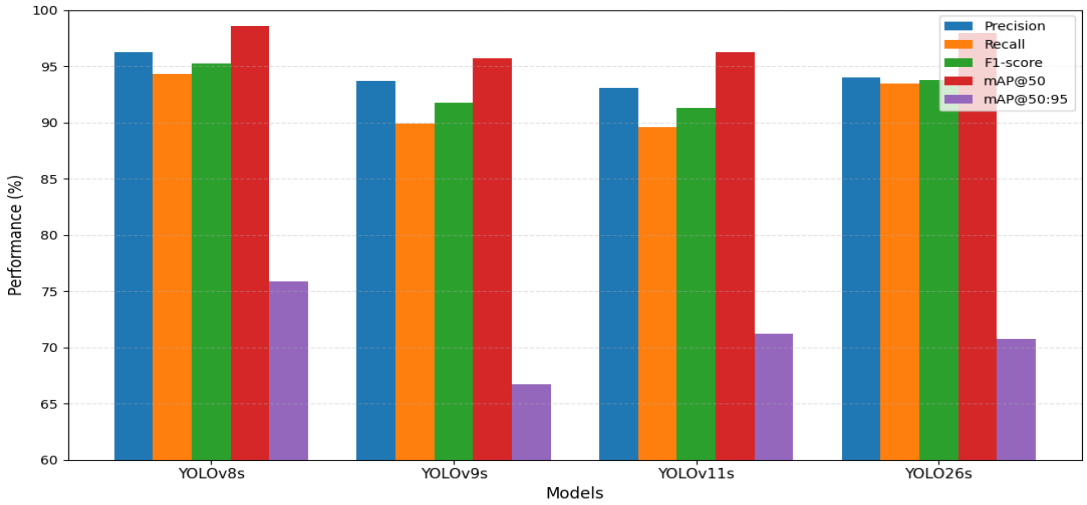


Figure 20: Illustrates the comparative performance of the four models.

Several observations can be drawn from the experimental results:

YOLOv8s consistently outperformed all other models across every evaluation metric. YOLO26s achieved the second-best overall performance, requiring the fewest training epochs while maintaining high Precision and Recall. YOLOv11s improved localization accuracy over YOLOv9s, particularly under higher IoU thresholds. YOLOv9s exhibited the weakest performance, especially for complex defect localization.

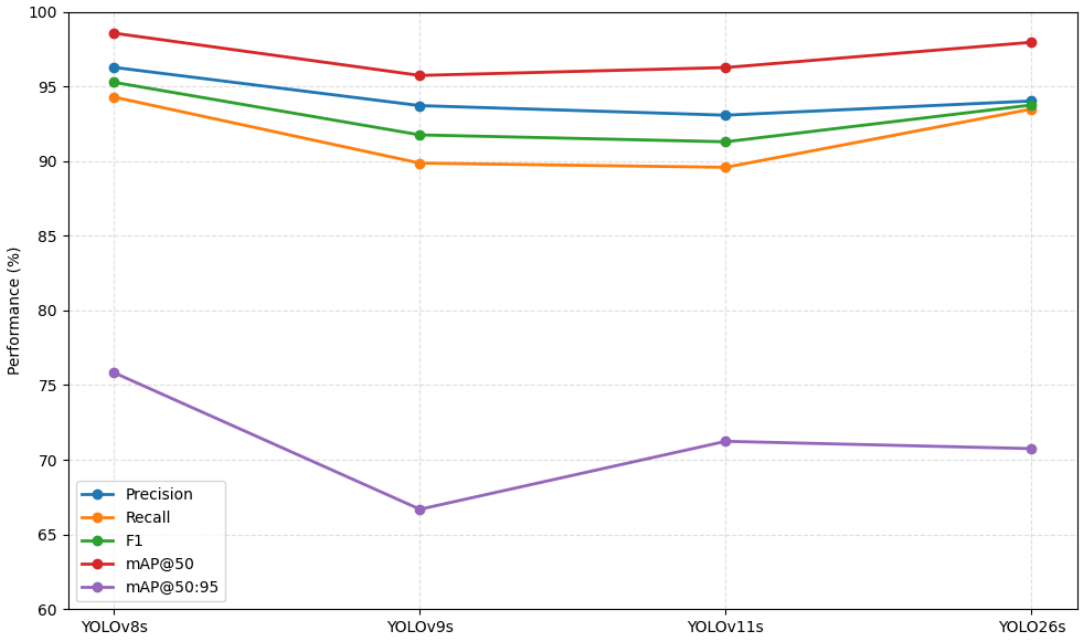


Figure 21: Illustrates the comparative performance of the four models in line charts.

The superior performance of YOLOv8s can be attributed to its balanced backbone architecture, efficient feature aggregation, and optimized detection head, which enable accurate detection of both large and fine-grained garment defects.

YOLO26s demonstrates promising characteristics for industrial deployment due to its combination of high accuracy and faster convergence. This reduces computational cost while preserving competitive detection performance.

5.3 Qualitative and Quantitative Performance Analysis of YOLOv8s

Figure 22 presents the comprehensive performance evaluation of the YOLOv8s model, including training samples, learning curves, representative detection results, and the normalized confusion matrix. The results demonstrate that YOLOv8s effectively learns discriminative features for garment defect detection while maintaining stable convergence throughout the training process.

Training and Convergence Analysis

The learning curves indicate that the bounding-box loss, classification loss, and Distribution Focal Loss (DFL) decrease steadily as training progresses, confirming successful optimization of the detection network. Similarly, the validation losses exhibit continuous reductions without noticeable divergence from the training losses, indicating that the model generalizes well to unseen samples and shows minimal evidence of overfitting.

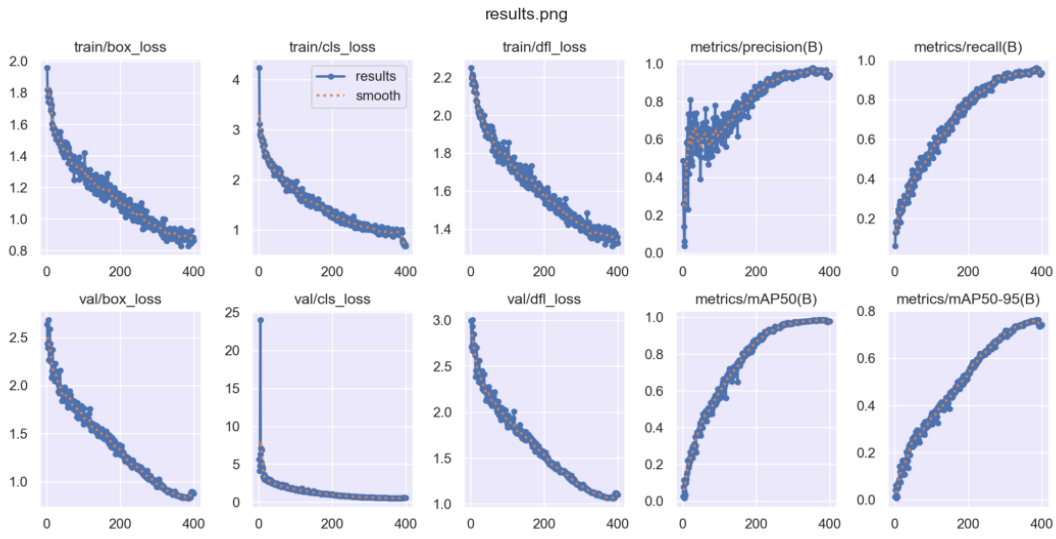


Figure 22: The comprehensive performance evaluation of the YOLOv8s model

The evaluation metrics further support the convergence behavior. Both precision and recall increase consistently during training and stabilize near the final epochs, while mAP@50 approaches approximately 0.99 and mAP@50–95 reaches nearly 0.76. These results indicate excellent localization accuracy and robust object detection performance across multiple IoU thresholds.

Detection Performance on Sample Images

The qualitative prediction results demonstrate that YOLOv8s successfully detects multiple garment defect categories, including hole, spot, knot slub, and yarn, while correctly identifying defect-free fabric samples. The predicted bounding boxes accurately enclose the defective regions, and the confidence scores indicate strong prediction certainty for most detected objects.

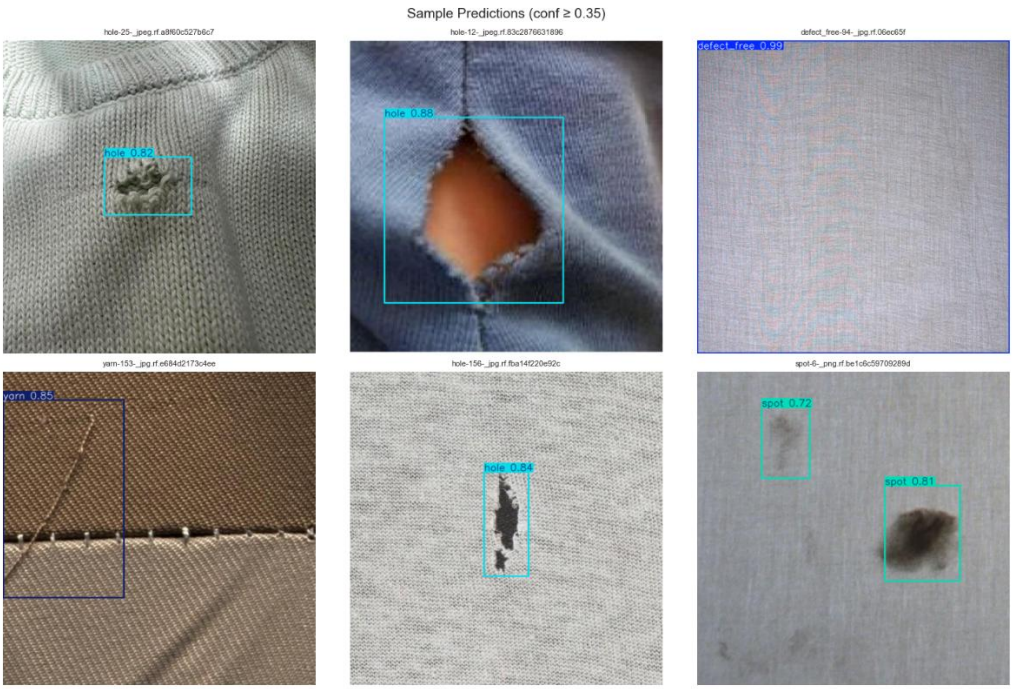


Figure 23: YOLOv8s successfully detects multiple garment defect categories.

The model performs particularly well on distinct defects such as holes and spots, where the defective regions possess clear visual boundaries. In contrast, fine-textured defects such as yarn and knot slub remain comparatively more challenging because of their irregular appearance, elongated structures, and similarity to surrounding fabric textures. Nevertheless, the model is able to localize these difficult defects with reasonable confidence, demonstrating its capability to learn subtle visual characteristics.

Normalized Confusion Matrix Analysis

The normalized confusion matrix illustrates the classification performance of YOLOv8s across all defect categories. The majority of samples are concentrated along the principal diagonal, indicating high classification accuracy and effective discrimination among garment defect classes.

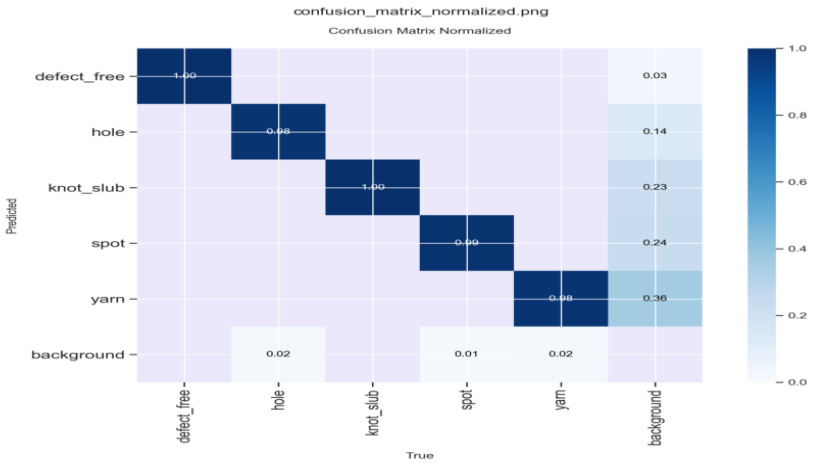


Figure 24: YOLOv8s normalized confusion matrix.

The defect-free category exhibits the highest classification accuracy, with nearly all samples correctly recognized. Likewise, hole, spot, and knot slub achieve high true-

positive rates with very limited confusion between classes. The primary source of prediction error originates from the yarn class, where a proportion of samples are incorrectly classified as background. This behavior is expected because yarn defects are generally thin, elongated, and possess lower visual contrast than other defect types.

Overall Performance Assessment

Overall, the qualitative and quantitative analyses demonstrate that YOLOv8s is highly effective for automated garment defect detection. The model exhibits stable convergence, excellent localization capability, and reliable classification performance across multiple defect categories. The prediction examples confirm its ability to accurately identify both isolated and subtle defects under varying fabric textures, while the normalized confusion matrix further validates its strong generalization capability.

Among the evaluated lightweight YOLO models, YOLOv8s consistently provides the best overall detection performance, achieving the highest mAP values, superior per-class detection accuracy, and the lowest level of inter-class confusion. These characteristics make YOLOv8s a strong candidate for deployment in real-time industrial garment quality inspection systems, where high detection accuracy, fast inference, and robust performance across diverse defect types are essential.

5.4 Discussion

The experimental findings indicate that all four lightweight YOLO architectures are capable of detecting garment defects with high accuracy. However, noticeable differences exist in their ability to localize difficult defects such as yarn and knot slub, which possess irregular textures and low visual contrast.

YOLOv8s achieved the most balanced performance across all defect categories, exhibiting stable convergence, minimal overfitting, and the highest overall detection accuracy. The confusion matrices further revealed that most prediction errors were associated with subtle defects being classified as background rather than confusion between defect classes.

YOLO26s showed comparable performance while requiring fewer training epochs, suggesting greater training efficiency. This characteristic makes it particularly attractive for real-time smart manufacturing systems where computational resources and training time are important considerations.

Although YOLOv11s demonstrated better localization capability than YOLOv9s, both models exhibited reduced performance when detecting fine-textured defects. Future improvements may focus on incorporating attention mechanisms, transformer-based feature extraction, or advanced data augmentation strategies to improve the detection of subtle garment defects.

Overall, the experimental results demonstrate that lightweight YOLO architectures provide an effective solution for automated garment quality inspection, with YOLOv8s emerging as the most reliable model for deployment in industrial environments.

Chapter 6

Conclusion and Limitations

This research presented an automated garment defect detection framework based on deep learning and computer vision using four lightweight YOLO models: YOLOv8s, YOLOv9s, YOLOv11s, and YOLO26s. The models were evaluated using Precision, Recall, F1-score, mAP@50, and mAP@50:95 to assess their suitability for industrial garment inspection.

Experimental results demonstrated that YOLOv8s achieved the best overall performance, obtaining a Precision of 96.27%, Recall of 94.28%, F1-score of 95.27%, mAP@50 of 98.56%, and mAP@50:95 of 75.83%. The model consistently outperformed the remaining architectures in both localization and classification tasks, making it the most effective solution for automated garment defect detection.

YOLO26s also produced competitive results while requiring fewer training epochs, indicating greater training efficiency. Although YOLOv11s and YOLOv9s achieved satisfactory detection accuracy, their performance on challenging defects such as yarn and knot slub remained comparatively lower.

The proposed deep learning framework demonstrates significant potential for replacing manual visual inspection in garment manufacturing by providing accurate, consistent, and real-time defect detection. The findings support the adoption of lightweight YOLO models as practical solutions for intelligent quality control in modern textile industries.

Limitations

Despite the encouraging experimental results, this study has several limitations:

The dataset contains only five garment defect categories, which may not represent the full range of defects encountered in industrial production.

Some defect classes, particularly yarn and knot slub, exhibit significant visual similarity to normal fabric textures, making accurate detection challenging.

The experiments were conducted under controlled imaging conditions, whereas industrial environments may introduce varying illumination, fabric orientation, and background noise.

Only lightweight YOLO architectures were evaluated. Other state-of-the-art detectors such as RT-DETR, EfficientDet, or transformer-based models were not investigated.

The computational efficiency was evaluated primarily through training convergence rather than real-time deployment on embedded industrial hardware.

Future research should investigate larger and more diverse textile datasets, advanced attention mechanisms, synthetic data augmentation, domain adaptation techniques, and deployment on edge AI devices to improve robustness in real-world manufacturing environments.

Chapter 7

Future Work

Future research should focus on expanding the garment defect dataset by incorporating a wider variety of fabric types, colors, textures, lighting conditions, and additional defect categories to improve the model's generalization capability in real-world manufacturing environments. Since the experimental results showed that fine-textured defects such as yarn and knot slub remain the most challenging to detect, future studies may explore advanced feature extraction techniques, including attention mechanisms, transformer-based architectures, multi-scale feature fusion, and enhanced data augmentation methods to improve the detection accuracy of subtle and low-contrast defects. Furthermore, evaluating recently developed object detection models and comparing them with the lightweight YOLO architectures could provide further insights into achieving higher detection performance. Another important direction is the deployment and validation of the proposed framework in real industrial production lines. Future work should investigate implementation on edge computing devices such as NVIDIA Jetson or other embedded AI platforms to achieve low-latency, real-time defect inspection with reduced computational requirements. In addition, integrating the defect detection system with industrial cameras, conveyor belt inspection systems, robotic sorting mechanisms, and manufacturing execution systems (MES) would enable a fully automated garment quality control process.

Bibliography

[1] A. Rasheed, B. Zafar, A. Rasheed, N. Ali, M. Sajid, S. H. Dar, U. Habib, T. Shehryar, and M. T. Mahmood, “Fabric Defect Detection Using Computer Vision Techniques: A Comprehensive Review,” Mathematical Problems in Engineering, vol. 2020, Article ID 8189403, 2020.

[2] D. Mathew and N. C. Brintha, “Deep-GD: Deep Learning based Automatic Garment Defect Detection and Type Classification,” International Journal of Electrical and Electronics Research, vol. 12, no. 1, pp. 41-47, 2024.

[3] A. A. K., E. J. Fernando, D. Kumar, and A. Gnanaselvi, “Automated Fabric Defect Detection Using Deep Learning Techniques,” International Journal of Novel Research and Development, vol. 9, no. 4, 2024.

[4] H. Lee, R. Grosse, R. Ranganathan, and A. Y. Ng, "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations," in Proceedings of the 26th Annual International Conference on Machine Learning, ACM, 2009, pp. 609–616.

[5] MathWorks, "Deep Learning with MATLAB," MATLAB Expo 2018, 2018.

[6] MathWorks, "Introducing Deep Learning with MATLAB," Deep Learning E-Book, MathWorks.

[7] CompleteGate, "Deep Machine Learning: LeNet and AlexNet CNN Architecture," [Online]. Available: https://www.completegate.com/.

[8] A. Berg, J. Deng, and L. Fei-Fei, "Large Scale Visual Recognition Challenge 2010," ImageNet, 2010. [Online]. Available: www.imagenet.org/challenges.

[9] F. Li, J. Johnson, and S. Yueng, "Lecture 9: CNN Architectures," Stanford University, May 2017.

[10] J. Sánchez and F. Perronnin, "High-dimensional signature compression for large-scale image classification," in 2011 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2011, pp. 1665–1672.

[11] MathWorks, "Image Category Classification Using Deep Learning," Computer Vision Toolbox Documentation. [Online]. Available: https://in.mathworks.com/help/vision/.

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Communications of the ACM, May 2015.

[13] MathWorks, "Deep Learning for Image Classification," Deep Learning Toolbox Documentation. [Online]. Available: https://in.mathworks.com/help/nnet/.

[14] P. V. V. Kishore, S. R. C. Kishore, and M. V. D. Prasad, "Conglomeration of hand shapes and texture information for recognizing gestures of Indian sign language using feed forward neural networks," International Journal of Engineering and Technology, vol. 5, no. 5, pp. 3742–3756, 2013.

[15] P. V. V. Kishore, M. V. D. Prasad, C. R. Prasad, and R. Rahul, "4-Camera model for sign language recognition using elliptical Fourier descriptors and ANN," in Proceedings of SPACES 2015 - IEEE International Conference on Signal Processing and Communication Engineering Systems, 2015, pp. 34–38.

Appendix

Appendix 1

Table 9: Addressing of COs, Knowledge Profile (K), and Complex Engineering Problem (EP)

CO	Knowledge Profile (K)	Complex Engineering Problem (EP)
CO1	**(i) Background [K1, K2, K3]K1:** Basic understanding of garment defects, textile materials, and quality inspection processes.**K2:** Mathematical foundations of deep learning, including convolution, feature extraction, pooling, and optimization algorithms.**K3:** Knowledge of computer vision, object detection, and CNN-based deep learning models.	**(i) Background [EP1]EP1:** Requires understanding of object detection architectures (YOLO) and computer vision techniques for garment defect detection [K3]. Knowledge of image preprocessing, annotation, and dataset preparation [K4]. Understanding industrial quality inspection requirements and textile manufacturing processes [K8].**(ii) Research Problem [EP6]EP6:** Detecting visually similar defects under varying lighting conditions requires domain knowledge, dataset analysis, and iterative model optimization.
CO2	**(i) Related Works [K8]**Comprehensive review of existing garment defect detection methods using computer vision and deep	**(i) Related Works [EP1]EP1:** Requires investigation of recent literature, state-of-the-art object detection algorithms, and industrial

CO	Knowledge Profile (K)	Complex Engineering Problem (EP)
	learning. Review of YOLO-based object detection models, Faster R-CNN, SSD, EfficientDet, and transformer-based detection approaches. Analysis of recent research on automated textile quality inspection.	inspection systems [K8]. Knowledge of transfer learning, model optimization, and deployment techniques [K4, K6].**(ii) Objectives [EP2, EP6, EP7]EP2:** Develop an accurate and real-time garment defect detection system.**EP6:** Improve industrial quality control by reducing manual inspection.**EP7:** Train and compare multiple YOLO architectures to identify the best-performing model.
CO3	**(i) Problem Analysis [K1–K4]K1:** Knowledge of garment defect characteristics and textile manufacturing defects.**K2:** Understanding optimization algorithms, backpropagation, and evaluation metrics.**K3:** Knowledge of deep learning-based object detection architectures.**K4:** Image preprocessing techniques including resizing, augmentation, normalization, and annotation.	**(i) Problem Analysis [EP1, EP2, EP3, EP6, EP7]EP1:** Selecting an appropriate object detection model for small and complex defects.**EP2:** Designing suitable preprocessing and augmentation strategies for defect images.**EP3:** Comparing multiple YOLO architectures to determine optimal performance.**EP6:** Validating detection performance for practical industrial inspection.**EP7:** Developing a complete workflow including preprocessing, training, testing, and evaluation.
CO4	**(i) Design & Implementation [K5]K5:** Knowledge of designing an automated defect detection workflow, model training, evaluation, and deployment using deep learning frameworks.	**(i) Design & Implementation [EP1, EP2, EP3, EP6, EP7]EP1:** Designing an end-to-end object detection pipeline using YOLO architectures.**EP2:** Managing large image datasets and GPU-based model training.**EP3:** Preserving fine defect features during preprocessing.**EP5:** Following standard software engineering and machine learning development practices.**EP7:** Implementing modular preprocessing, training, validation, testing, and performance comparison stages.

CO	Knowledge Profile (K)	Complex Engineering Problem (EP)
CO5	**(i) Materials and Tools [K6]K6:** Implementation using Python, Ultralytics YOLO, OpenCV, NumPy, PyTorch, Streamlit, and CUDA-enabled GPU computing.	**(i) Materials & Tools [EP1, EP2, EP4, EP5]EP1:** Requires proficiency in deep learning frameworks and computer vision libraries.**EP2:** Handling GPU memory limitations and computational complexity during training.**EP4:** Optimizing inference speed for real-time industrial deployment.**EP5:** Ensuring ethical use of AI systems and reproducible experimental procedures.
CO6	**(i) Social and Industrial Impact [K7]K7:** Understanding the industrial, economic, and ethical implications of AI-driven automated quality inspection systems.	**(i) Social & Industrial Impact [EP2, EP5, EP6]EP2:** The proposed system supports human inspectors by improving inspection accuracy and productivity rather than replacing skilled workers.**EP5:** Promotes responsible AI deployment, transparency, and ethical engineering practices.**EP6:** The developed system has the potential to improve production efficiency, reduce manufacturing waste, and support Industry 4.0 initiatives in the textile sector.